

欢迎来到未来 [01-Raw/26生成式软件工程/NJU]

原视频: <https://www.bilibili.com/video/BV1pb8o6yE8f> · 课程合集:
<https://space.bilibili.com/202224425/lists/8942017> · 讲者: 蒋炎岩 (NJU 《生成式软件工程》2026)

全课主线

讲者想让听众相信: 计算机史反复把昂贵能力变成日用品, 今天的 AI/agent 正在把同样的事推到学习、工作和简历评价上, 因此旧式“古法编程”和只靠成绩/标准简历的路已经不够。讲者要听众转向生成式软件工程: 开始用 agent, 把需求编译成可复用的代码或 skill, 并同时补上软件工程里 intention→specification→implementation 的 gap。最终要能回答: 为什么导师要招你, 而不是买一个 200 美元的 Claude Max。

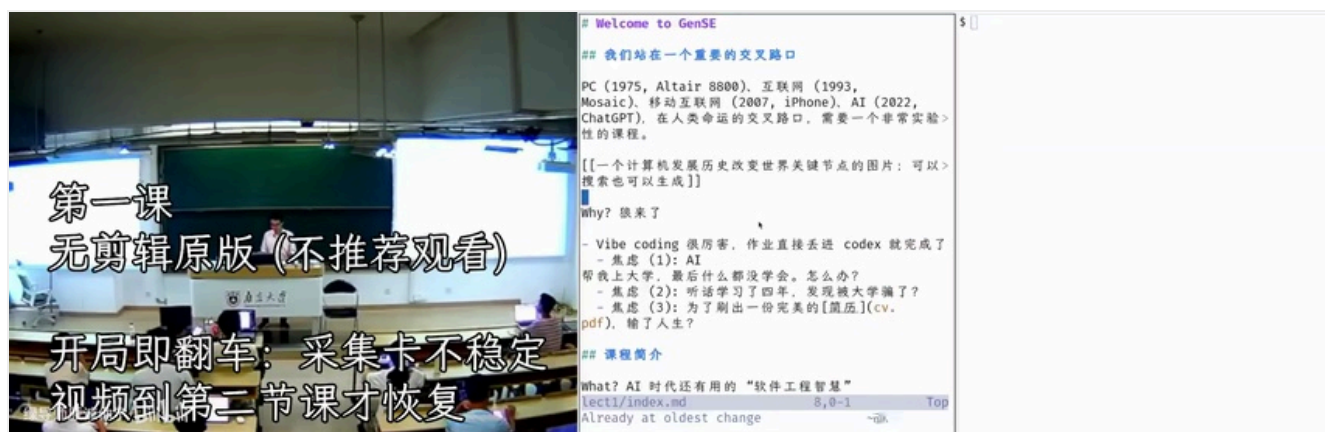
1. 讲者从计算机史切入, 主张计算机史上最主要的变化是把昂贵能力变成日用品; 他用 PC→互联网→移动互联网→ChatGPT 的例子说明连接、支付、AI 能力不断进入普通人日常, 因此今天处在“学什么、怎么学”都要重新思考的交叉路口。(第1部分)
2. 在“AI 替我上大学”的闭环里, 老师用豆包出题、学生用豆包做作业, 只有学生没受训练; coding 工具还能直接处理作业, 所以问题变成到底还学不学编程, 讲者答“既应该也不应该”。(第1、2部分)
3. 讲者诊断两类焦虑: AI 能答对所有开卷考试、听话认真读完四年也只学会 AI 都会的东西; 同时 AI 让完美简历无法区分你和竞争者, 问题变成“什么能让你脱颖而出”。因此他提出用 vibe coding 的快乐缓解焦虑。(第2、3部分)
4. 课程定位为生成式软件工程, 但首先是软件工程课, 目标是帮学生建立驾驭人工智能的能力; 课堂以案例、松散讨论为主, 讲者还用 Markdown 作 source of truth、让 AI 生成 slides/图片, 并把课堂表演、录像复盘、切片回灌 code 连成探索“杀死大学”和课程数字永生的路线。(第3、4、5部分)
5. 课程 policy 是禁止古法编程, token 自费但用便宜且够智能的模型; 因为模型辅助产出无法统一评分, 所以申请通过/不通过, 最终价值落到简历项目“我在生成式软件工程里实现了 ABCDE”。(第5、6、7部分)
6. 技术理解上, 讲者把大语言模型说成概率分布/next-token, agent 因长上下文、推理、coding/ReAct 出现, 能调用 bash 并循环; agent 从去年不可用到今年可用, 但 next-

token 可能效率低，未来路线不可预测。(第7、8、9部分)

7. 实际使用 agent 的原则是“开始，现在就开始用”和“随时问他”，并用 AGENTS.md 等配置给 agent 上下文；例子包括一句话让 AI 装 Debian U 盘、用 QEMU 验证、配置终端和 tiling WM，由此学习方式变成判断 AI 能力边界、带 AI 做它做不到的事。(第10、11部分)
8. 讲者把 agent 接到个人流程：邮件草稿、Apple Reminders、看书/论文、B 站长视频总结等；会用 agent 学习的人与不会用的人效率差距 10—100 倍，于是竞争又回到简历和作品。(第12、13、14部分)
9. 他进一步说 agent 生态已达临界点：USB 故障可由 agent 软件重连，“空白平台+API”可复现，Skill Hub/开源脚本让能力复用极轻松；像 Unix 管道一样组合 skill/脚本会形成知识飞轮，个人能力被成百上千倍放大。(第15、16、17、18、19部分)
10. 但是，AI 单点强不等于产品能成：产品是系统性工程，AI 会生成不可维护代码；教务系统、QR code、批卷与受控机器等案例显示 intention、specification、implementation 之间有 gap，模型因 reward 变成“猜模型”和做题家，先扩大需求、缺好架构，所以软件工程仍未死。(第20、21、22、23、24、25、26、27、28部分)
11. 结论落在保研问题上：古法编程已死，AI 完成单个任务已无人能敌，但软件工程暂时未死；讲者借用 GPT-5.6 的回答收束为能找到值得解决的问题、把未知拆成便宜实验、用证据让坏方向停下、对长期结果负责。(第28、29部分)

讲者的结论/号召是：古法编程已死、软件工程暂未死，学生应开始用 agent、把需求编译成可复用代码或 skill，并用作品回答“为什么招你而不是买 200 美元 Claude Max”。

第 1 部分





交叉路口的主线：计算机史是把昂贵能力变成日用品

- 讲者开场：在 Neovim 中按 F 打开浏览器，然后正式上课。
- 听众是 Computer Science 同学；讲者从计算机发展史切入。
- 讲者认为，计算机史上经历多次重要变革，最主要的变化是把昂贵的能力变成日用品。
- 因此今天处在“非常重要的交叉路口”：上课方式、该学什么，都需要重新思考。
- 讲者说明课堂是松散讨论/交流，之后会把课程重新切片、整理。

例证链：PC → 互联网 → 移动互联网 → ChatGPT

- 1975 年第一台 PC：Altair 8800（转录作“PCGR泰尔 8800”），在图片左边。图片由 AI 生成，有明显 AI 痕迹（原文“AI word”），而且模型不算特别强；讲者说没关系。
- PC → 互联网：互联网连接所有人。
 - 讲者个人经历：小学快毕业时装了 ICQ；腾讯山寨版叫 OICQ，后来改成 QQ。
 - 那时可以随机加世界另一端的人；讲者加到 native speaker，对方一下子发很多英语、打得非常快，自己完全无法理解——因为当时刚学一点英语，只会 hello、Good morning。
 - （整理者串联）这段经历显示：互联网把“连接世界另一端的人”变成普通人可随手尝试的事，和“能力日用品化”的主线一致。
- 互联网 → 移动互联网：讲者印象很深，一夜之间所有菜场卖菜的人都有支付宝和微信两个二维码；很短时间里生活方式被改变。
- 2022 年：ChatGPT：讲者称它是人类历史上增长最快的产品。
- 逻辑链：计算机变革反复把昂贵能力日用品化 → 这些例子显示连接、支付、AI 能力进入普通人的日常（整理者串联） → 当前再次来到“学什么、怎么学”的交叉路口。

“AI替我上大学”：闭环中只有学生没受训练

- 在这个背景下，讲者决定开 web coding 课。
- 起因是流传的说法“AI 替我上大学”。讲者认为这个说法很对。
- 讲者给出的闭环：
 1. 老师用 豆包 出题；
 2. 大家用 豆包 做作业；
 3. 整个闭环里，只有学生没有受到训练。
- 现在 coding 工具很厉害：作业可以直接处理。
 - 对比：操作系统作业曾折磨大家，讲者认为大家在痛苦折磨中获得提升。
 - 现在：把作业链接丢进 Codex，不需要复制粘贴；甚至可以不给链接，只说“蒋老师的操作系统课第八个实验，能不能帮我实现”，就结束了。
- 由此问题变成：这样的时代还怎么办？到底应不应该学这些编程？

讲者的答案是“既应该也不应该”

- 对“到底应不应该学这些编程”，讲者先给出答案：既应该也不应该。

第 2 部分



Why? 狼来了

作业完成了，学习发生了吗？

Vibe coding 很厉害，作业直接丢进 codex 就完成了。

◆ 过去，作品 = 能力留下的痕迹；现在，同一份作品可能对应完全不同的能力。

“◆ 如果 Codex 能交出满分作业，满分究竟证明了谁的什么能力？”



南京航空航天大学

教育背景

姓名: [REDACTED] 学号: [REDACTED] 毕业时间: 2027.6

学分平均分: [REDACTED] 年级排名: [REDACTED] 英语水平: CET4: [REDACTED], CET6: [REDACTED]

课程成绩: 操作系统 [REDACTED], 计算机组成原理 [REDACTED], 高等数学 [REDACTED], 概率论 [REDACTED], 大学英语 [REDACTED]

荣誉奖项: [REDACTED]

科研经历

1. 面向多模态情感分析的分阶段扩散与混合专家模型研究 [REDACTED], 第一作者 2025.6-2026.1

项目简介: 针对多模态情感分析中文本模态通常为主导模态, 但文本噪声严重影响模型鲁棒性的问题, 提出认知启发式两阶段多模态情感分析框架 [REDACTED], 该方法创新性地在文本模态引入扩散模型进行语义重建, 并结合混合专家机制, 实现对噪声文本的渐进增强与鲁棒多模态融合。

项目职责: 提出认知启发式“先学习后纠正”的两阶段多模态情感分析框架 [REDACTED], 引入扩散模型进行文本语义重建以应对噪声问题; 设计并实现定制化 Hybrid Model 结构, 融合线性注意力与 softmax 注意力, 实现鲁棒的语义增强与多模态融合; 独立完成模型实现, 实验验证与消融分析。

项目成果: 产出学术论文 [REDACTED], 会议已正式接受 [REDACTED]

2. 面向细粒度多模态情感分析的心理引导图构建与混合专家方法 [REDACTED] 在投, 第一作者 2025.10-至今

项目简介: 针对现有方法在声学、视觉细粒度情感细粒度建模中扁平化线索遗漏的问题, 本项目在心理学实验数据驱动与多模态感知机制的共同作用下, 以“强化心理学情感感知规律差异, 抑制共性、抑制冗余”为原则整合视觉线索信息, 并对情感语义进行解耦并针对不同目标和模态实施选择增强, 从而提升模型的细粒度情感分析能力与可解释性。

项目职责: 负责本文问题提出与核心方法 [REDACTED] 设计实现, 包括构建数学公式心理语义规律转化为图神经网络, 以及扩散模型、多专家交互解耦与池化特征选择的设计实现; 同时负责主要实验、可解释性分析、结果整理以及论文写作等。

项目成果: 产内科学术论文 [REDACTED] 正在投



南京航空航天大学

教育背景

姓名: [REDACTED] 学号: [REDACTED] 毕业时间: 2027.6

学分平均分: [REDACTED] 年级排名: [REDACTED] 英语水平: CET4: [REDACTED], CET6: [REDACTED]

课程成绩: 操作系统 [REDACTED], 计算机组成原理 [REDACTED], 高等数学 [REDACTED], 概率论 [REDACTED], 大学英语 [REDACTED]

荣誉奖项: [REDACTED]

科研经历

1. 面向多模态情感分析的分阶段扩散与混合专家模型研究 [REDACTED], 第一作者 2025.6-2026.1

项目简介: 针对多模态情感分析中文本模态通常为主导模态, 但文本噪声严重影响模型鲁棒性的问题, 提出认知启发式两阶段多模态情感分析框架 [REDACTED], 该方法创新性地在文本模态引入扩散模型进行语义重建, 并结合混合专家机制, 实现对噪声文本的渐进增强与鲁棒多模态融合。

项目职责: 提出认知启发式“先学习后纠正”的两阶段多模态情感分析框架 [REDACTED], 引入扩散模型进行文本语义重建以应对噪声问题; 设计并实现定制化 Hybrid Model 结构, 融合线性注意力与 softmax 注意力, 实现鲁棒的语义增强与多模态融合; 独立完成模型实现, 实验验证与消融分析。

项目成果: 产出学术论文 [REDACTED], 会议已正式接受 [REDACTED]

2. 面向细粒度多模态情感分析的心理引导图构建与混合专家方法 [REDACTED] 在投, 第一作者 2025.10-至今

项目简介: 针对现有方法在声学、视觉细粒度情感细粒度建模中扁平化线索遗漏的问题, 本项目在心理学实验数据驱动与多模态感知机制的共同作用下, 以“强化心理学情感感知规律差异, 抑制共性、抑制冗余”为原则整合视觉线索信息, 并对情感语义进行解耦并针对不同目标和模态实施选择增强, 从而提升模型的细粒度情感分析能力与可解释性。

项目职责: 负责本文问题提出与核心方法 [REDACTED] 设计实现, 包括构建数学公式心理语义规律转化为图神经网络, 以及扩散模型、多专家交互解耦与池化特征选择的设计实现; 同时负责主要实验、可解释性分析、结果整理以及论文写作等。

项目成果: 产内科学术论文 [REDACTED] 正在投



南京航空航天大学

教育背景

姓名: [REDACTED] 学号: [REDACTED] 毕业时间: 2027.6

学分平均分: [REDACTED] 年级排名: [REDACTED] 英语水平: CET4: [REDACTED], CET6: [REDACTED]

课程成绩: 操作系统 [REDACTED], 计算机组成原理 [REDACTED], 高等数学 [REDACTED], 概率论 [REDACTED], 大学英语 [REDACTED]

荣誉奖项: [REDACTED]

科研经历

1. 面向多模态情感分析的分阶段扩散与混合专家模型研究 [REDACTED], 第一作者 2025.6-2026.1

项目简介: 针对多模态情感分析中文本模态通常为主导模态, 但文本噪声严重影响模型鲁棒性的问题, 提出认知启发式两阶段多模态情感分析框架 [REDACTED], 该方法创新性地在文本模态引入扩散模型进行语义重建, 并结合混合专家机制, 实现对噪声文本的渐进增强与鲁棒多模态融合。

项目职责: 提出认知启发式“先学习后纠正”的两阶段多模态情感分析框架 [REDACTED], 引入扩散模型进行文本语义重建以应对噪声问题; 设计并实现定制化 Hybrid Model 结构, 融合线性注意力与 softmax 注意力, 实现鲁棒的语义增强与多模态融合; 独立完成模型实现, 实验验证与消融分析。

项目成果: 产出学术论文 [REDACTED], 会议已正式接受 [REDACTED]

2. 面向细粒度多模态情感分析的心理引导图构建与混合专家方法 [REDACTED] 在投, 第一作者 2025.10-至今

项目简介: 针对现有方法在声学、视觉细粒度情感细粒度建模中扁平化线索遗漏的问题, 本项目在心理学实验数据驱动与多模态感知机制的共同作用下, 以“强化心理学情感感知规律差异, 抑制共性、抑制冗余”为原则整合视觉线索信息, 并对情感语义进行解耦并针对不同目标和模态实施选择增强, 从而提升模型的细粒度情感分析能力与可解释性。

项目职责: 负责本文问题提出与核心方法 [REDACTED] 设计实现, 包括构建数学公式心理语义规律转化为图神经网络, 以及扩散模型、多专家交互解耦与池化特征选择的设计实现; 同时负责主要实验、可解释性分析、结果整理以及论文写作等。

项目成果: 产内科学术论文 [REDACTED] 正在投



南京航空航天大学

教育背景

姓名: [REDACTED] 学号: [REDACTED] 毕业时间: 2027.6

学分平均分: [REDACTED] 年级排名: [REDACTED] 英语水平: CET4: [REDACTED], CET6: [REDACTED]

课程成绩: 操作系统 [REDACTED], 计算机组成原理 [REDACTED], 高等数学 [REDACTED], 概率论 [REDACTED], 大学英语 [REDACTED]

荣誉奖项: [REDACTED]

科研经历

1. 面向多模态情感分析的分阶段扩散与混合专家模型研究 [REDACTED], 第一作者 2025.6-2026.1

项目简介: 针对多模态情感分析中文本模态通常为主导模态, 但文本噪声严重影响模型鲁棒性的问题, 提出认知启发式两阶段多模态情感分析框架 [REDACTED], 该方法创新性地在文本模态引入扩散模型进行语义重建, 并结合混合专家机制, 实现对噪声文本的渐进增强与鲁棒多模态融合。

项目职责: 提出认知启发式“先学习后纠正”的两阶段多模态情感分析框架 [REDACTED], 引入扩散模型进行文本语义重建以应对噪声问题; 设计并实现定制化 Hybrid Model 结构, 融合线性注意力与 softmax 注意力, 实现鲁棒的语义增强与多模态融合; 独立完成模型实现, 实验验证与消融分析。

项目成果: 产出学术论文 [REDACTED], 会议已正式接受 [REDACTED]

2. 面向细粒度多模态情感分析的心理引导图构建与混合专家方法 [REDACTED] 在投, 第一作者 2025.10-至今

项目简介: 针对现有方法在声学、视觉细粒度情感细粒度建模中扁平化线索遗漏的问题, 本项目在心理学实验数据驱动与多模态感知机制的共同作用下, 以“强化心理学情感感知规律差异, 抑制共性、抑制冗余”为原则整合视觉线索信息, 并对情感语义进行解耦并针对不同目标和模态实施选择增强, 从而提升模型的细粒度情感分析能力与可解释性。

项目职责: 负责本文问题提出与核心方法 [REDACTED] 设计实现, 包括构建数学公式心理语义规律转化为图神经网络, 以及扩散模型、多专家交互解耦与池化特征选择的设计实现; 同时负责主要实验、可解释性分析、结果整理以及论文写作等。

项目成果: 产内科学术论文 [REDACTED] 正在投



教育背景

毕业时间: 2027.6
学分平均分: [REDACTED] 年级排名: [REDACTED] 英语水平: CET4 [REDACTED], CET6 [REDACTED]
课程成绩: 操作系统 [REDACTED], 计算机组成原理 [REDACTED], 高等数学 [REDACTED], 概率论 [REDACTED], 大学英语 [REDACTED]
荣誉奖项: [REDACTED]

科研经历

1. 面向多模态情感分析的分阶段扩散与混合专家模型研究 [REDACTED] · 第一作者 2025.6-2026.1
项目简介: 针对多模态情感分析中文本模态通常为主导模态, 但文本噪声严重影响模型鲁棒性的问题, 提出认知启发式的两阶段多模态情感分析框架 [REDACTED]。该方法创新性地在本模态引入扩散模型进行语义重建, 并结合混合专家机制, 实现对噪声文本的渐进增强与鲁棒多模态融合。
项目职责: 提出认知启发式“先学习后纠正”的两阶段多模态情感分析框架 [REDACTED]。引入扩散模型进行文本语义重建以应对噪声问题; 设计并实现定制化 Hybrid MoE 结构, 融合线性注意力与 softmax 注意力, 实现鲁棒的语义增强与多模态融合; 独立完成模型实现, 实验验证与消融分析。
项目成果: 产出学术论文 [REDACTED] 会议已正式接受
2. 面向细粒度多模态情感分析的心理学引导图构建与混合专家方法 [REDACTED] 在投 · 第一作者 2025.10-至今



学分平均分: [REDACTED] 年级排名: [REDACTED] 英语水平: CET4 [REDACTED], CET6 [REDACTED]
课程成绩: 操作系统 [REDACTED], 计算机组成原理 [REDACTED], 高等数学 [REDACTED], 概率论 [REDACTED], 大学英语 [REDACTED]
荣誉奖项: [REDACTED]

科研经历

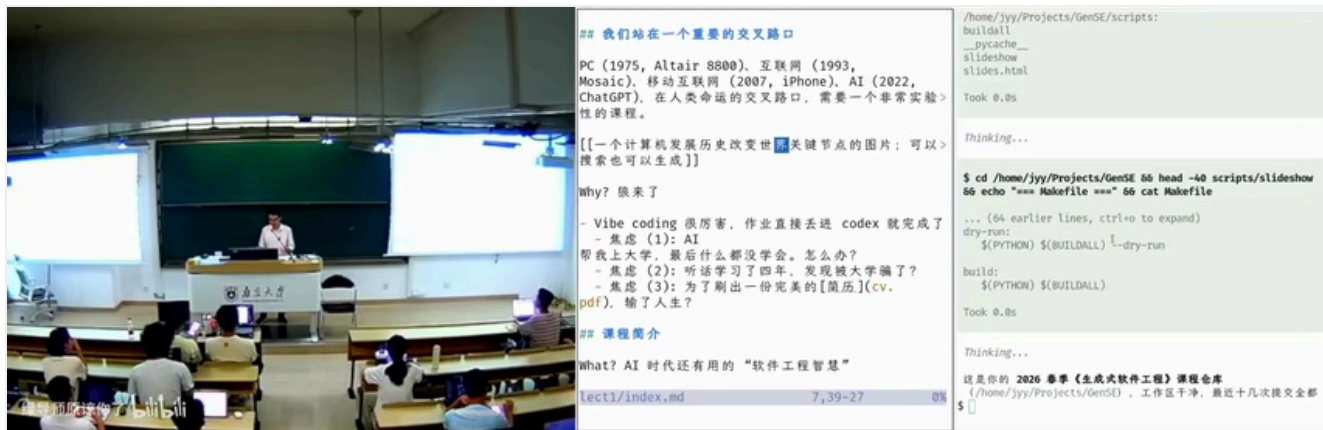
1. 面向多模态情感分析的分阶段扩散与混合专家模型研究 [REDACTED] · 第一作者 2025.6-2026.1
项目简介: 针对多模态情感分析中文本模态通常为主导模态, 但文本噪声严重影响模型鲁棒性的问题, 提出认知启发式的两阶段多模态情感分析框架 [REDACTED]。该方法创新性地在本模态引入扩散模型进行语义重建, 并结合混合专家机制, 实现对噪声文本的渐进增强与鲁棒多模态融合。
项目职责: 提出认知启发式“先学习后纠正”的两阶段多模态情感分析框架 [REDACTED]。引入扩散模型进行文本语义重建以应对噪声问题; 设计并实现定制化 Hybrid MoE 结构, 融合线性注意力与 softmax 注意力, 实现鲁棒的语义增强与多模态融合; 独立完成模型实现, 实验验证与消融分析。
项目成果: 产出学术论文 [REDACTED] 会议已正式接受
2. 面向细粒度多模态情感分析的心理学引导图构建与混合专家方法 [REDACTED] 在投 · 第一作者 2025.10-至今
项目简介: 针对现有方法在声学、视觉细粒度情感线索建模中扁平化线索蒸馏的问题, 本项目在心理学实验邻接矩阵与多跳传播机制的共同作用下, 以“强化心理学的情感类别共现差异, 抑制共现、偶然、冗余线索”为原则整合视听线索信息, 并对传播后的语义进行解耦并对不同目标模态实施选择性增强, 从而提升模型的细粒度情感分析能力与可解释性。
项目职责: 我负责本文问题提出与核心方法 [REDACTED] 的设计实现, 包括构建数学公式将心理学规律转化为邻接矩阵, 以及扩散机制、多专家交互解耦与视角特定路由增强的设计实现; 同时负责主要实验、可解释性分析、结果整理以及论文主体撰写。
项目成果: 产出学术论文 [REDACTED] 会议在投
3. [REDACTED] · 通过置信度引导的层次化情感推理 [REDACTED] 核心参与者, 预投稿 2026.1-至今



- 项目职责: 提出认知启发式“先学习后纠正”的两阶段多模态情感分析框架 [REDACTED]。引入扩散模型进行文本语义重建以应对噪声问题; 设计并实现定制化 Hybrid MoE 结构, 融合线性注意力与 softmax 注意力, 实现鲁棒的语义增强与多模态融合; 独立完成模型实现, 实验验证与消融分析。
项目成果: 产出学术论文 [REDACTED] 会议已正式接受
2. 面向细粒度多模态情感分析的心理学引导图构建与混合专家方法 [REDACTED] 在投 · 第一作者 2025.10-至今
项目简介: 针对现有方法在声学、视觉细粒度情感线索建模中扁平化线索蒸馏的问题, 本项目在心理学实验邻接矩阵与多跳传播机制的共同作用下, 以“强化心理学的情感类别共现差异, 抑制共现、偶然、冗余线索”为原则整合视听线索信息, 并对传播后的语义进行解耦并对不同目标模态实施选择性增强, 从而提升模型的细粒度情感分析能力与可解释性。
项目职责: 我负责本文问题提出与核心方法 [REDACTED] 的设计实现, 包括构建数学公式将心理学规律转化为邻接矩阵, 以及扩散机制、多专家交互解耦与视角特定路由增强的设计实现; 同时负责主要实验、可解释性分析、结果整理以及论文主体撰写。
项目成果: 产出学术论文 [REDACTED] 会议在投
3. [REDACTED] · 通过置信度引导的层次化情感推理 [REDACTED] 核心参与者, 预投稿 2026.1-至今
项目简介: 针对 LLMs 在情感推理中 (1) 难以区分语义歧义情感及 (2) 易受冗余背景干扰的挑战, 本项目提出了一种免训练的情感感知增强框架 [REDACTED]。通过置信度引导的由粗到细推理实现多级分类标签, 并动态剔除无关视觉 Token 实现视觉聚焦, 显著提升了模型在复杂场景下的情感推理精度。
项目职责: (1) 我设计了两阶段推理框架: 第一阶段利用模型最后一层注意力图量化视觉特征与情感指令的关联度, 并计算 Logits 的方差作为置信度; 第二阶段首先剔除低关联度的视觉 Token 来消除冗余信息干扰, 随后通过置信度动态调整推理路径, 从而缓解语义相似类别的推断混淆; (2) 我在 EmoSet 数据集上展开了样本性能基准测试, 消融实验与可视化分析。
4. 完整对开模型中的因果特征发现 [REDACTED] 核心参与者 2026.1-2026.2



1. 面向多模态情感分析的分阶段扩散与混合专家模型研究 [REDACTED] · 第一作者 2025.6-2026.1
项目简介: 针对多模态情感分析中文本模态通常为主导模态, 但文本噪声严重影响模型鲁棒性的问题, 提出认知启发式的两阶段多模态情感分析框架 [REDACTED]。该方法创新性地在本模态引入扩散模型进行语义重建, 并结合混合专家机制, 实现对噪声文本的渐进增强与鲁棒多模态融合。
项目职责: 提出认知启发式“先学习后纠正”的两阶段多模态情感分析框架 [REDACTED]。引入扩散模型进行文本语义重建以应对噪声问题; 设计并实现定制化 Hybrid MoE 结构, 融合线性注意力与 softmax 注意力, 实现鲁棒的语义增强与多模态融合; 独立完成模型实现, 实验验证与消融分析。
项目成果: 产出学术论文 [REDACTED] 会议已正式接受
2. 面向细粒度多模态情感分析的心理学引导图构建与混合专家方法 [REDACTED] 在投 · 第一作者 2025.10-至今
项目简介: 针对现有方法在声学、视觉细粒度情感线索建模中扁平化线索蒸馏的问题, 本项目在心理学实验邻接矩阵与多跳传播机制的共同作用下, 以“强化心理学的情感类别共现差异, 抑制共现、偶然、冗余线索”为原则整合视听线索信息, 并对传播后的语义进行解耦并对不同目标模态实施选择性增强, 从而提升模型的细粒度情感分析能力与可解释性。
项目职责: 我负责本文问题提出与核心方法 [REDACTED] 的设计实现, 包括构建数学公式将心理学规律转化为邻接矩阵, 以及扩散机制、多专家交互解耦与视角特定路由增强的设计实现; 同时负责主要实验、可解释性分析、结果整理以及论文主体撰写。
项目成果: 产出学术论文 [REDACTED] 会议在投
3. [REDACTED] · 通过置信度引导的层次化情感推理 [REDACTED] 核心参与者, 预投稿 2026.1-至今
项目简介: 针对 LLMs 在情感推理中 (1) 难以区分语义歧义情感及 (2) 易受冗余背景干扰的挑战, 本项目提出了一种免训练的情感感知增强框架 [REDACTED]。通过置信度引导的由粗到细推理实现多级分类标签, 并动态剔除无关视觉 Token 实现视觉聚焦, 显著提升了模型在复杂场景下的情感推理精度。
项目职责: (1) 我设计了两阶段推理框架: 第一阶段利用模型最后一层注意力图量化视觉特征与情感指令的关联度, 并计算 Logits 的方差作为置信度; 第二阶段首先剔除低关联度的视觉 Token 来消除冗余信息干扰, 随后通过置信度动态调整推理路径, 从而缓解语义相似类别的推断混淆; (2) 我在 EmoSet 数据集上展开了样本性能基准测试, 消融实验与可视化分析。



“既应该也不应该”：能力仍需要，但具体怎么做不清楚

- 讲者回应“到底应不应该学编程”：答案是既应该也不应该。
- 应该：有些能力你还是需要。
- 但应该怎么做，讲者说“我们不知道”。
- 讲者观察到大家处在非常焦虑的状态，随后展开两类焦虑。

焦虑一：AI 帮你上大学，毕业后你可能第一个被 AI 取代

- 讲者设想：每门课都很好；如果每门课都开卷考试，AI 都能答题且全对。
- 但毕业出来发现：第一个被 AI 取代。
- 讲者的推理链：AI 能答对所有开卷考试 → 你每门课都很好、全答对 → 毕业后仍可能被 AI 取代。
- （整理者串联）因此“成绩好”不再自动等于“有不可替代的能力”。

焦虑二：你认真听话读完四年，却只学会 AI 都会的东西

- 讲者描述另一类同学：延续中学时很听老师话的状态——老师说什么就做什么，每堂课认真听，每次作业认真做。
- 四年毕业时发现“被大学骗了”：确实学会了点什么，但学会的东西 AI 都会做。
- 结果同样迷茫。
- （整理者串联）与焦虑一不同：不是你让 AI 替你学，而是你亲自按规则学，但学到的东西仍可被 AI 替代。

保研海投最痛：AI 让“完美简历”也无法区分你和竞争者

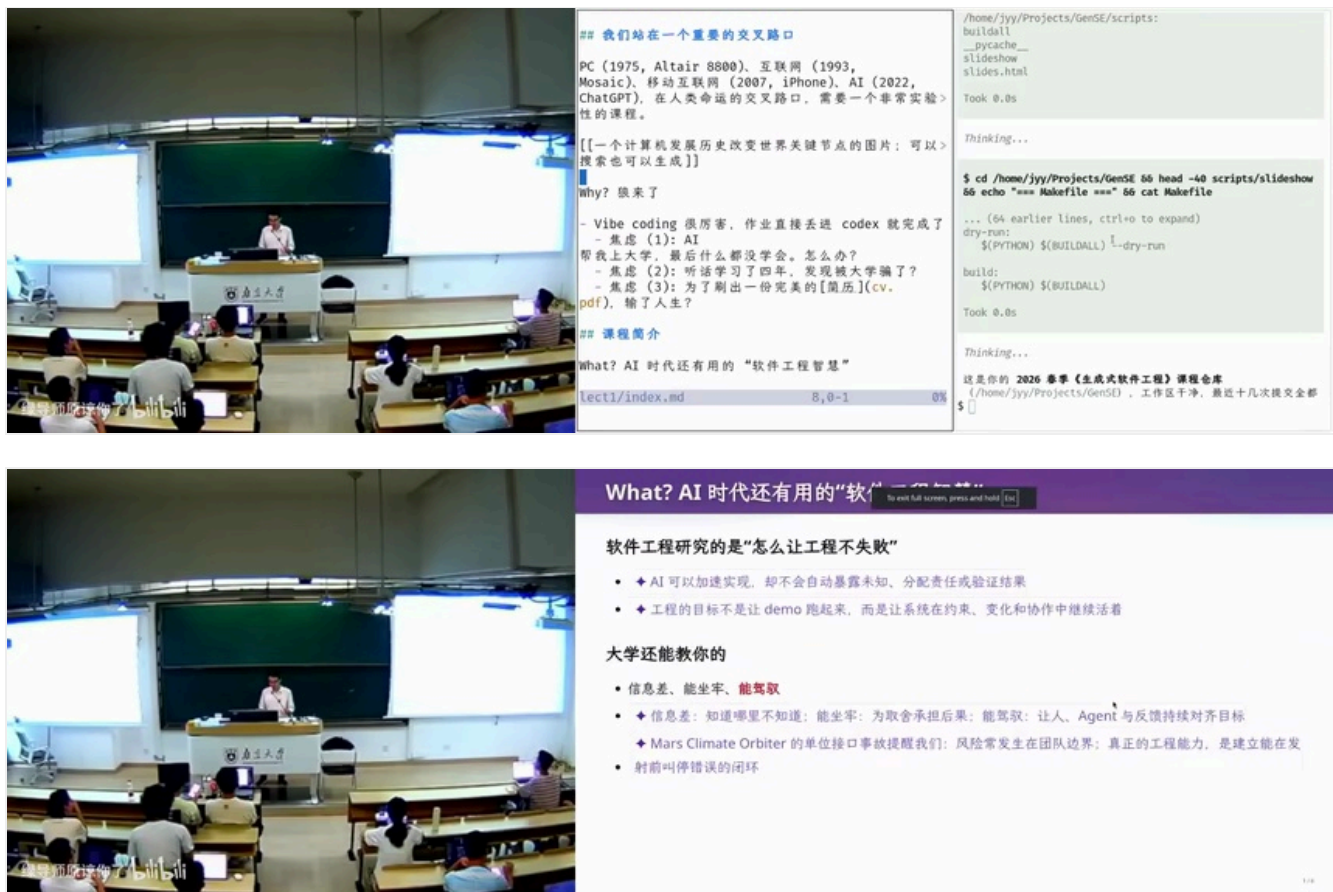
- 讲者认为同学们现在最切实、最痛苦的事是**保研**：投很多简历，海投很多地方。
- 讲者自己也收到过很多这样的简历，并认为很糟糕。他展示一份简历：
 - 讲者用 AI 处理这份简历：他告诉 AI 有一份简历要课堂展示，需要去掉个人信息；AI 把字符替换成 **Unicode 方块**，复制粘贴会看到黑色方块字符，用来隐藏隐私。讲者说这些操作“都是 AI 做的，我没有干任何事”。
 - 简历表面很完美：有 **CCF 期刊列表**上的论文，状态是 **accepted**。
 - 讲者提到“大家都说**在投**（原转录作“再投”）就等于不存在”，因为你看不到。
- 讲者对这份简历的评价：**零分简历**。
 - 理由：没有额外信息；甚至没有 **GitHub 链接**、没有个人主页、没有 **preprint**。
 - 讲者不知道这位同学发生了什么，但认为这样一份简历非常糟糕；更糟糕的是他收到过无数份这样的简历，可能在座同学的简历也多多少少长这样。
- 讲者由此推出：
 1. AI 能批量生成看起来很完美的简历。
 2. 于是所有人都有一大堆实习、一大堆奖项、一大堆论文、一大堆在投。
 3. **你和你的竞争者已经无法区分。**
 4. 问题变成：**什么能让你从这些简历里脱颖而出？**
 5. 如果刚才那份简历都被讲者认为是零分，那应该怎样向未来导师展示自己？

讲者的缓解方案：用 web coding 的快乐缓解焦虑

- 讲者相信在座同学非常焦虑。
- 他的“好消息”是：希望用这门课来缓解这种焦虑情绪，更多带大家还原一种快乐——**web coding 的快乐**。
- 讲者说自己在 web coding 中得到很多快乐，然后在这个快乐当中.....（转录在此中断）。

小结：讲者先诊断 AI 让课程成绩和标准简历都变得不可区分，再提出用 web coding 的快乐缓解焦虑。

第 3 部分



我们站在一个重要的交叉路口

PC (1975, Altair 8800)、互联网 (1993, Mosaic)、移动互联网 (2007, iPhone)、AI (2022, ChatGPT)。在人类命运的交叉路口。需要一个非常实验性的课程。

[[一个计算机发展历史改变世界关键节点的图片：可以>搜索也可以生成]]

why? 烦来了

- Vibe coding 很厉害。作业直接丢进 codex 就完成了
- 焦虑 (1): AI 帮我上大学。最后什么都没学会。怎么办?
- 焦虑 (2): 听话学习了四年。发现被大学骗了?
- 焦虑 (3): 为了刷出一份完美的[简历](cv.pdf)。输掉了人生?

课程简介

What? AI 时代还有用的“软件工程智慧”

lect1/index.md 8,0-1 0%

```
/home/jyy/Projects/GenSE/scripts:
buildall
__pycache__
slideshow
slides.html
Took 0.0s

Thinking...

$ cd /home/jyy/Projects/GenSE && head -40 scripts/slideshow
&& echo "=== Makefile ===" && cat Makefile

... (64 earlier lines, ctrl+o to expand)
dry-run:
$(PYTHON) $(BUILDALL) |>-dry-run

build:
$(PYTHON) $(BUILDALL)

Took 0.0s

Thinking...

这是你的 2026 春季《生成式软件工程》课程仓库
{ /home/jyy/Projects/GenSE }。工作区干净，最近十几次提交全部
```

What? AI 时代还有用的“软件工程智慧”

软件工程研究的是“怎么让工程不失败”

- ◆ AI 可以加速实现，却不会自动暴露未知、分配责任或验证结果
- ◆ 工程的目标不是让 demo 跑起来，而是让系统在约束、变化和协作中继续活着

大学还能教你的

- ◆ 信息差、能坐牢、**能驾驭**
- ◆ 信息差：知道哪里不知道；能坐牢：为取舍承担后果；能驾驭：让人、Agent 与反馈持续对齐目标
- ◆ Mars Climate Orbiter 的单位接口事故提醒我们：风险常发生在团队边界；真正的工程能力，是建立能在发射前叫停错误的闭环

课程首先想还原 vibe coding 的快乐，再把快乐接到软件工程上

- 讲者面对/提到大家的焦虑情绪，认为课程更多是想带大家还原一种快乐：**vibe coding** 的快乐（转写作“web coding”，按语境应为 vibe coding）。
- 他给出自己的理由：他自己在 vibe coding 中得到很多快乐。
- 这条链是讲者明说的：
 1. 先进入 **vibe coding** 的快乐；
 2. 在快乐中，学生能一定程度上理解软件工程是怎么工作的；
 3. 能做出有趣的东西；
 4. 也能让简历显得更好看。
- 讲者个人看法：有一篇发表论文可能是一个**加分项**；但对很多其他人来说，可能还不一定。

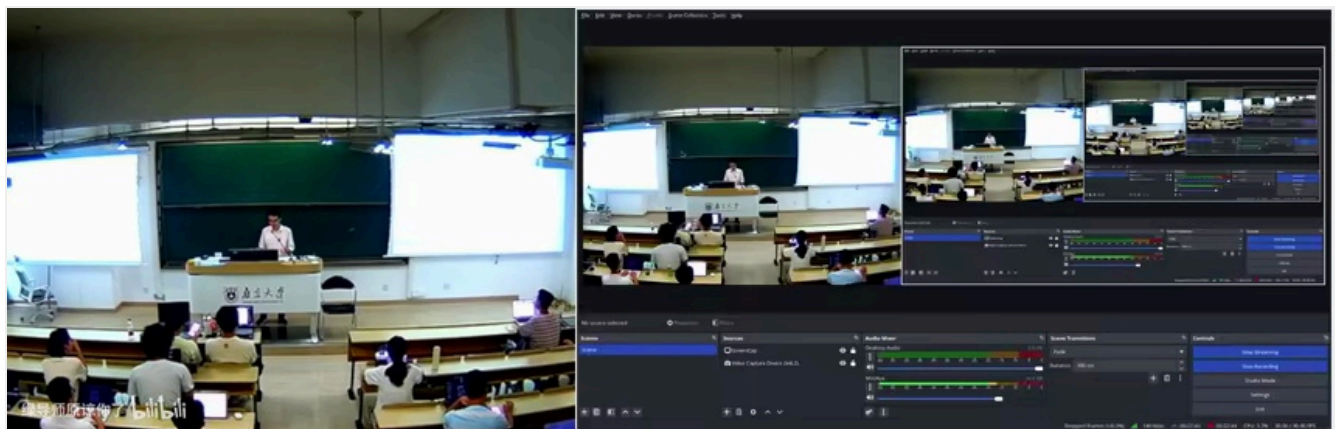
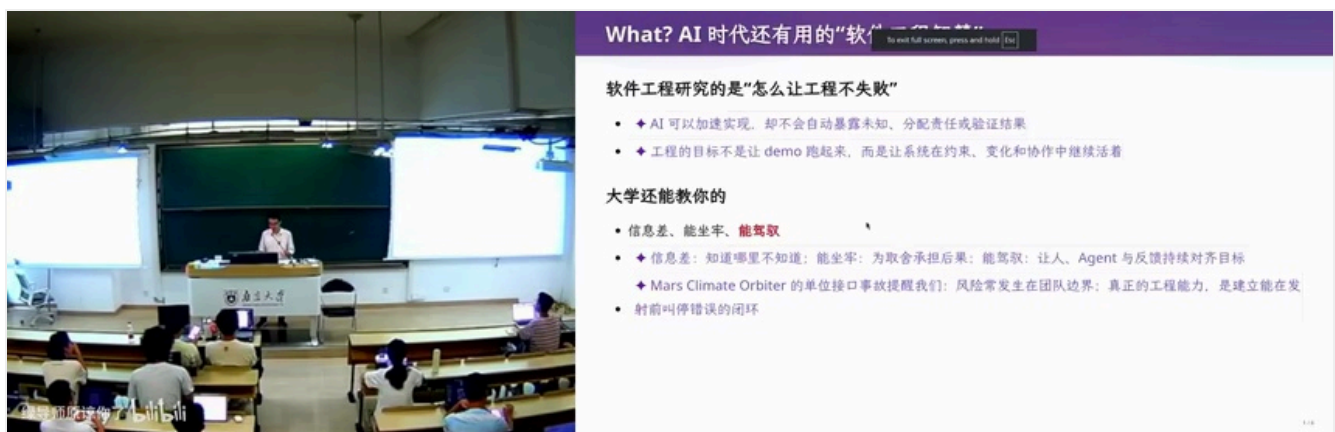
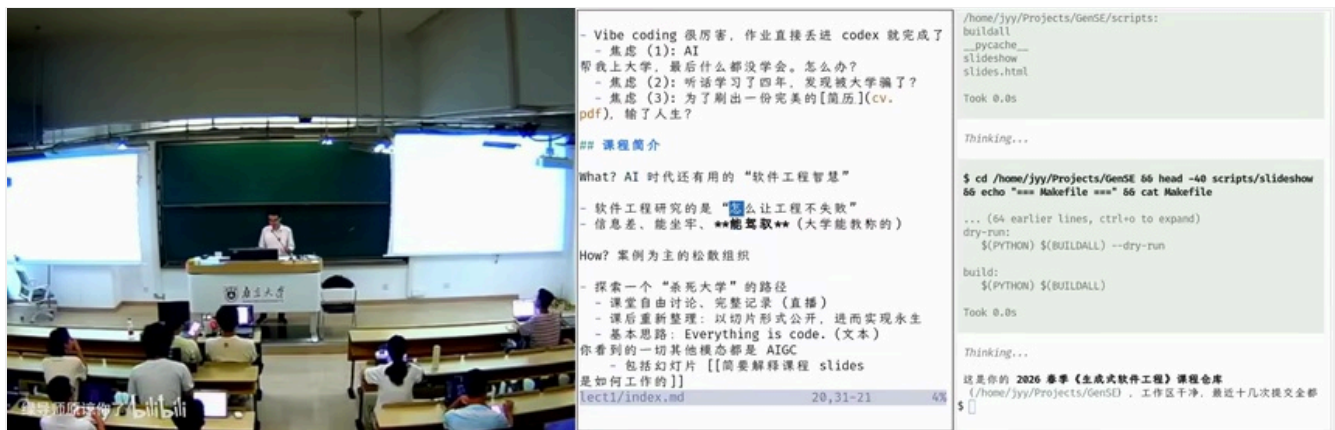
生成式软件工程首先是一门软件工程课

- 课程题目叫 **生成式软件工程**，所以讲者说它本质上是**软件工程课**。
- 软件工程做什么？讲者的定义很简单：**怎么样让软件工程不失败**，也就是**怎么样造出一个软件**。
- 讲到人与 AI 的比较时，讲者举了一个说法：“你可以坐牢，AI 不能”【原文如此】。
- 讲者认为，大学毕业时比较重要的能力，首先是 **信息差**：
 - 你有别人不知道的信息，才能做别人做不到的事。
 - 讲者认为信息差是人类社会进步所必需的，不能说可悲。
 - 学校里的例子：你拥有一份别人都没有的**往年试卷**，就能在竞争中胜出。
 - 社会上还有更多“不能说的信息差”广泛存在。
- 除信息差外，讲者更强调：大学真正教你的是**怎么样成为一个更好的人**，即**能够驾驭别人驾驭不了的东西**；毕业时你可以说“我是一个更好的人，我有一个别人没有的能力”。
- 因此，这门课试着帮大家建立**驾驭人工智能的能力**。

上课方式：案例为主、组织松散；想讨论来线下，听课不必到课

- 讲者希望课程以**案例为主**，组织比较**松散**。
 - 如果你想和讲者讨论、**challenge** 他（“老师你这样做不对”），欢迎来线下。
 - 如果你是**听课型**、希望从课程里获取知识，讲者完全建议**既不用到课，也不用看直播**。
 - 原因是：他今年想做一个非常有趣的尝试。
 - 他展示自己的 slides：按 **F5** 可以播放；F5 播放的 slides 和刚才那个 **markdown** 有点不一样。
 - 左边是他上课之前写的**讲义**：他会先想好今天大概什么时间讲什么、举什么例子——相当于**约束好 AI 做什么**。
 - 当你看到 F5 播放 slides 时，会看到它和 markdown 的区别。
-

第 4 部分



用 AI 约束课堂内容：先定“这个时间讲什么、举什么例子”

- 讲者今天上课时先要判断：到这个时间点应该讲什么、举什么样的例子。
- 这相当于约束好 AI 做什么：不是让 AI 自由发挥，而是先给出目标和边界。
- 当用 F5 播放 slides 时，紫色部分就是 **GPT-5.6 (原音 GBT5.6)** 给讲者“添油加醋”加上去的部分。

紫色内容来自 `mix lights`：从 Markdown 演讲稿生成有趣内容

- 讲者有一个提示词/skill，叫 `mix lights`（原音技能名）。
- 他可以从自己写好的一个 **Markdown 演讲稿**出发，让 AI 添油加醋，做点有趣的事情。
- 例子：他有一个类似 **AI 耿同学**的 skill，会让它生成 3 个 **sub-agent**，并给它们赋予不同的 **personality**。
- 讲者认为这个非常有效果：有的 agent 干这个，有的干那个，而且它们在**隔离的上下文**里工作。
- 最后由 **主 H**（原音）回来汇总。

多 sub-agent 再汇总：用算力换智力

- 讲者明确说，这样他就可以**通过算力来换智力**：付出更多算力，就获得更多智力。
- 逻辑链：多个 sub-agent 分别在不同人格、隔离上下文里工作 → 主 H 汇总 → 用更多算力换更高智力。

生成 slides 不是一次成型：多版提示词 + 另一模型盲评

- 讲者对比了几个版本的提示词。
- 每一个版本提示词，他都会重新生成一次 slides。
- 最后还有一个 **review**：由另外一个模型**盲评**到底哪一个生成的 slides 质量比较高。
- 他基本调到了一个相对质量比较高的 slides。
- 因此，看到的图片全部都是生成的，所有图片都是 AI 生成的。

目标：探索真正“杀死大学”的路径

- 讲者说，他希望探索一条能够真正“杀死大学”的路径（原音如此）。
- 他的理由方向是：如果学生是来学习的，那更高效的方法是什么；作为老师，应该思考怎样把知识以更好的方式呈现给大家。
- 他的答案是：**上课是一种表演**。
- 他把老师比作演员：要调动大家的情绪，上课时跟大家交流，并根据**预先设定好的剧本**去做一场演出。

课后必须复盘：即兴不完美，录像可回灌代码

- 演出完后，讲者认为有一件很重要的事：**review 今天的表演**。
- 原因：有时今天举的例子可能不是特别恰当；回过头来，也许讲另外一个案例，大家能够更加感同身受。
- 上课有些部分是**即兴**的，但即兴意味着其实不完美。
- 因为课程被录像，他可以在课后把录像切片、**重新整理成片段**。
- 他说自己是从 **content as code** 来的，**everything is code**。
- 于是可以把上课录像——这次表演——重新回到 code 上，把它变得更丰富。
- 讲者由此说：这不就实现了**数字永生**吗？也就是实现课程的**数字永生**。
- 逻辑链：课堂录像 → 切片整理 → 回到 code → 丰富课程 → 课程数字永生。

这套 slides 的工作流：Markdown 是 source of truth，其他全部 AIGC

- 讲者再次强调 **everything is code**：世界里面的一切都可以用代码生成，包括桌子、椅子、电脑，以及 CAD (computer-aided design)。
- slides 的工作机制是：刚才看到的 **Markdown 是 source of truth**。
- 其他所有东西——**图片、幻灯片**——全部都是 **AIGC**。
- （整理者串联）把这些环节合起来，就是：Markdown 源 → AI 添油加醋生成 → 图片与 slides 全部 AIGC → 课堂表演录像 → 回灌 code 继续丰富课程。

第 5 部分



Everything is code

这套 slides 如何工作？

- 文本是 source of truth：你看到的其他模态都是 AIGC
- ✦ 完整讲义提供上下文 → 选中的 snippet 划定边界 → Agent 扩写 → Markdown slides → 浏览器渲染
- ✦ 生成内容统一标记为 aigc：逻辑改变时修改源码，图片、网页和视频切片都可以重新生成

要“永生”的不是一份录像

- ✦ 保存可编辑、可 diff、可复现的源代码与构建过程，而不只是最终像素
- 如果成功，就“永生”我的操作系统课

Markdown 是 source of truth，图片和幻灯片只是 AIGC 呈现

讲者用当前 slides 说明 workflow：

1. 刚才看到的 Markdown 是 source of truth。
2. 你看到的所有其他东西——图片、幻灯片——全部都是 AIGC。
3. 讲者会选中一个 snippet，用 agent 去扩写。
4. 扩写后变成 Markdown slides。
5. 再到浏览器里渲染出来。

→ 源是 Markdown；图片、幻灯片等只是由 Agent 扩写、生成并渲染出来的呈现层。

长期目标：把课程切片成可维护、可定制的程序

- 讲者希望用切片的方式得到一门完美的课。比如“今天要讲生成式软件工程”，它的要点是什么、逻辑是什么、第一节课应该怎么上。
- 这些内容要沉淀成一个程序。
- 这个程序也许现在由讲者维护；也许有一天讲者不维护了——比如他不再上系统课/操作系统课——他会把自己的 B 站操作系统课录像整理成这样的切片。
- 然后可以由全世界的人来维护它。
- 任何人想要定制“讲者的操作系统课”成什么样子，就可以把它定制成什么样子。
- 讲者认为：这不就把所有大学教课的方式给颠覆了嘛（转录作“巅峰”，按语境是颠覆）。
- 讲者认为这是一条可能实现的路线，所以会在这门课上做一个比较轻的尝试，也欢迎随时反馈。

课程 policy：先找回“上课耽误学习”的感觉，再要求禁止古法编程

- 讲者说这门课的 policy 是：希望重新找回同学们心中大学时那种“上课耽误学习”的感觉。
- 虽然可能还是要上课，但讲者推荐：第一个，你们可能可以不要上课，然后在这门课里实践。
- 讲者会带着大家实践，后面有很多真正实践的案例。
- 目的是让同学知道：在 AI 时代，学习方式变了，工作方式也变了；要尽快适应这个非常有趣的时代。

- 讲者的要求是：**禁止古法编程**。
- 讲者认为学生个个都是古法编程的好手：因为要考试、马上要保研、要机试；到时候把学生关小黑屋，看过讲者以前出的题会吓尿，然后赶快再去 **LeetCode** 刷题。
- 但讲者认为总有一天这个事情会改变。可能**自上而下传导不会那么快**；改变一个体系很难，但他非常想撕开一个小口子。
- 接着有几个 policy：第一个是 **token 自费**。

Token policy：自费，用便宜但够智能的模型跑通教学

- 讲者本来想去拉一些 **token 赞助**，但看到选课学生数量的时候就想还是算了；没必要再拉着脸去要好多好多 token，也比较麻烦。
- 讲者的个人观点：如果你们手上没有 token、没有付费 token，应该立即**退学**。
- 理由是：你可能确实不需要 **token maxing**（转录作 token maxine）。
- 在硅谷有一种文化：你要死命地烧 token。
- 讲者课上所有演示，除了 **slides 的生成**用一个更聪明的模型（讲者认为它能给更好的质量）以外，你看到的幻灯片、整个 lecture 里的东西，包括这些生成的 slides（转录作 S），全部都是 **deep pick vision flash**（保留原转录；指一个足够便宜但智能相对有一些的模型）。
- 也就是说，这门课会用**足够便宜、但有一定智能的模型**带大家完成。
- （整理者串联）这与“token 自费”的 policy 一致：不依赖赞助，也不要求所有人烧高价 token，而是用低成本模型跑通教学流程。

第 6 部分





课程 Policy: Token 自费

没有 token 的 CS 学生应该立即退学

- 个人观点
- 不需要 tokenmaxxing
- 课程课堂全程 deepseek-v4-flash
- ♦ Token 是 AI 时代的实验耗材：亲自使用，才能形成成本—质量—延迟意识
- ♦ 真正稀缺的不是 token，而是值得花 token 的问题、清晰约束和可靠验收



5/4

用便宜但够智能的模型，把 token 成本压低

- 讲者说，包括这些生成的内容在内，全部都是用 DeepSeek [原文“deep pick vision flash”] 这类模型完成的；他会在课上用一个“足够便宜、智能相对有一些”的模型带大家完成课程任务。
- 他知道大家是 CS 学生、是夜猫子，喜欢在夜里工作；DeepSeek 夜里比较便宜，所以他建议在“梁文峰/梁文谷时间”（原文如此）使用。白天要上课，晚上回去做作业时正好用。
- **Token 很重要。**讲者让每个同学去 DeepSeek 充 100 块；如果 token 有困难，可以联系他帮忙想办法。
- 逻辑链：便宜模型可用 → token 负担可控 → 学生能实际动手做生成式软件工程项目。
（整理者串联）

想用通过/不通过考核：因为无法统一评判“模型辅助产出”的质量

- 讲者在申请课程时，考核方式填的是 **通过/不通过**。
- 教务反馈：专业选修课一般不会这样做；通常“形势与政策”这类课才通过/不通过。
- 讲者说，现在还不知道评判标准是什么。
- 他的理由链：同一个任务，用更好的模型质量就更好；让 Claude [原文“cloud fable”] 来做，会比 DeepSeek Flash [原文“DP g vs flash”] 做得又快又好。讲者认为没法限制大家用什么模型，所以很难评判。
- 尤其这是一门 **web coding / project** 课程；讲者会有一些建议项目，之后更新到课程网站上告诉大家。

项目建议：做一个能和讲者邮箱通信的 email 管理系统

- 讲者建议实现自己的 **email 管理系统**，可以回复他的邮件。

- 他会用一个指定邮箱给学生发邮件，学生把邮件回回来，作为一个尝试。
- 形式是 **web code 服务**，跑在宿舍里一个一直常开的小电脑上。
- 讲者认为这是不错的实验；有同学会做复杂一点，有同学做简单一点。他可能会用自己的邮箱跟大家发邮件通信。

“做得好”的标准：分数不重要，简历上有什么才重要

- 讲者追问：到底什么叫做得好、什么叫做得不好、该给多少分？
- 讲者认为，在这个时代 **分数已经不那么重要**；真正重要的是最后找工作或保研时，投出的 **简历上有什么东西**。
- 他也说，为了走到那一刻，仍然需要比较好的 **GPA**，还得 **reward hacking**，还得 **overfit** 考试题目。但他认为最终检验大学阶段的是：你交出去的简历能不能带你到下一个更高的平台。
- 他举例：你敢不敢给 **DeepSeek [原文“DBP”]** 投简历？他们的 bar 比较高。

最终目标：简历里写“在生成式软件工程里实现了 ABCDE”

- 讲者希望这门课结束时，大家能在简历里写：我在 **生成式软件工程** 里实现了这个 **ABCDE**。
- 后面有很多链接，并且每个东西都做得非常漂亮。
- 终点：课程评价不靠统一分数，而靠可展示的项目作品和简历产出。（整理者串联）

小结：讲者用便宜模型降低 token 门槛，用 pass/fail 回避无法统一的评分，最终把课程价值落到可展示的简历项目上。

第 7 部分



课程 Policy: 尽量“通过 / 不通过”

不验收作业，不评分

- 有建议的实验和反馈
- 现在越来越难找到客观衡量标准
- “服从性考试没有意义”——大家都一样，职场见
- ♦ 不评分 / 不评价：去掉排名信号，把反馈留给下一次迭代

用作品回答“你成就了什么？”

- 希望大家能在简历里写：“在《生成式软件工程》课程中实现了 A, B, C, D, ... (github.io 链接)”
- 例子：我的主页
 - ♦ 最终证据不是分数，而是别人可以打开、运行、质疑并继续使用的作品：github.io 是一条可审计的成长轨迹
- 迹



PC (1975, Altair 8800), 互联网 (1993, Mosaic), 移动互联网 (2007, iPhone), AI (2022, ChatGPT), 在人类命运的交叉路口，需要一个非常实验性的课程。

[[一个计算机发展历史改变世界关键节点的图片：可以搜索也可以生成]]

why? 快来了

- Vibe coding 很厉害，作业直接丢进 codex 就完成了
- 焦虑 (1): AI 帮我上大学，最后什么都没学会。怎么办？
- 焦虑 (2): 听说学习了四年，发个 CV 被大学骗了？
- 焦虑 (3): 为了刷出一份完美简历 (cv, pdf)，输掉了人生？

课程简介

what? AI 时代还有用的“软件工程智慧”

- 软件工程研究的是“怎么让工程不失败”
- 信息差、能坐牢、**能驾驭** (大学能教你的)

lect1/index.md 13,44-33 3%

```
/home/jyy/Projects/GenSE/scripts:
buildall
__pycache__
slideshow
slides.html
Took 0.0s

Thinking...

$ cd /home/jyy/Projects/GenSE && head -40 scripts/slideshow
&& echo "" && Makefile "" && cat Makefile
... (64 earlier lines, ctrl+o to expand)
dry-run:
$(PYTHON) $(BUILDALL) --dry-run
build:
$(PYTHON) $(BUILDALL)
Took 0.0s

Thinking...

这是你的 2026 春季《生成式软件工程》课程仓库
(/home/jyy/Projects/GenSE)，工作区干净，最近十几次提交全部
$
```

简历叙事：生成式软件工程的项目要写成“我实现了 ABCDE”

- 讲者设想学生最后能在简历里写：我在生成式软件工程里实现了 ABCDE，后面附很多链接，每个东西都做得很漂亮。
- 关键不是只写“我学过”，而是每一项都能说清楚：我完成了设计、实现。
- 讲者认为，如果每个项目都做到这一点，本身就是很了不起的地方。

极端设想：电脑全程监控 → AI 评判你的大学四年

- 讲者提出一个可能未来：如果每个人的电脑上都安装监控，那么掌握 AI 的人就能知道你大学四年干过的所有事情，AI 就可以评判你度过了一个怎样的大学。
- 讲者认为这非常非常恐怖：因为你摸的每一条鱼、看的每一个不想让别人知道的小电影，都发生在电脑上，因此都可能被知道。

- 讲者说不知道人类社会什么时候会演进到那个程度，但主张**保持 open**；并称这是课程的 policy（原音似“polo 系”，识别不清），也算是课程开的一个头。
- （整理者串联）上一段讲的是 AI 评估简历里的项目成果，这一段把评估对象扩展到大学四年的行为过程：不只评估你“做出了什么”，还可能评估你“怎么度过时间”。

大语言模型：本质是一个学到的概率分布

- 讲者说今天不展开“什么是大语言模型”的完整原理，而是给一个使用者需要抓住的理解：**大语言模型就是一个概率分布**。
- 这与概率统计课上的抛硬币题没有本质区别：都是给定条件，问某种结果出现的概率。
- 使用方式：给模型一个很长的文本，比如一本书；这个长输入可以是文本，也可以是图像。书到末尾结束后，你还可以在后面再写几个字，例如：“能不能帮我总结一下这本书？”
- 接下来，大语言模型会不断预测下一个 **token 的概率**；每次输出可以理解成输出一个字符。
- 讲者认为：这个模型很大，见过世界上所有东西，训练它很难，所以现在 **frontier lab** 这么牛。

这门课站在使用者视角：不需要先懂内部原理

- 讲者强调，这门课是站在**使用者视角**上。
- 你们只需要知道存在一个东西叫大语言模型，可以打开 DVPC（识别不清）的网页/API 去体验。
- 讲者现场说要登录，又决定不登录；打开网页/API，问“什么是大语言模型、我怎么样体验它”，页面本身就会告诉你：你正在体验大语言模型。
- 所以，不需要先掌握大语言模型的内部知识，也可以开始使用它。

第一次被大模型“按在地上打”：GPT-4 给五岁孩子解释扫描电子显微镜

1. 时间是 2023 年，GPT-4 时代；讲者评价 GPT-4 是一个**幻觉非常严重**的模型。
2. 任务：给讲者五岁的孩子解释什么是**扫描电子显微镜**。

3. 当时的 prompt 思路是玩 **prompt engineering**：要求用朴素的语言、打一个合适的比方，帮助孩子理解。
4. 讲者有一天给五岁孩子讲故事，不知为何讲到了扫描电子显微镜，讲不下去了。
5. GPT-4 给了一个比方，讲者认为这个比方打得比自己还好。
6. 这个比方是 GPT-4 给出的，讲者让它写下来；比喻开头是：“你现在有一个山……”

第 8 部分



GPT-4 (2023): 给五岁孩子解释 SEM

不只是知道，而是为听众重编码

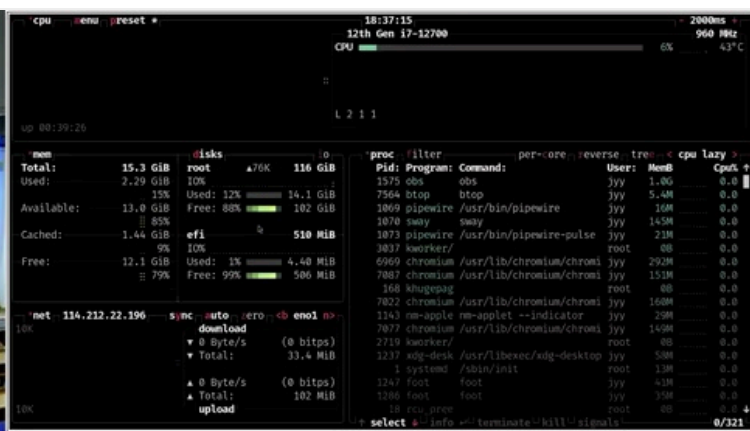
为我的五岁小朋友解释扫描电子显微镜 (SEM): 朴素的语言、打合适的比方，便于他理解本质原理。

“

◆ 把样品想成黑屋里的一小块月球地面。SEM 不用相机一次拍完整张照片，而拿一支看不见、细得不得了
“电子铅笔”，一行一行扫过表面。铅笔尖碰到哪里，那里就会蹦出或弹回一些电子；旁边的“小桶”把它们接
住，接得多就在屏幕上画亮一点，接得少就画暗一点。无数亮暗小点拼起来，我们就看见了肉眼看不到的表
面纹理。

- ◆ 因果骨架没有丢：电子束逐点扫描 → 探测二次/背散射电子 → 信号强弱映射成像素。
- ◆ SEM 不是“拿更强的相机放大”，而是扫描、测量、重建：图像像三维地形，也不等于直接测得三维高度。
- ◆ 产品跃迁不只是“知道 SEM”，而是能建立听众模型：检索系统检索语，LLM 可以成为知识与他人之间的认知接口。

2/4



“闭眼扔石头测山形”的比喻：用探测反馈像 ray tracing 一样反推不可见结构

- 讲者让模型写一个比方：你有一座山，但闭着眼看不到它的形状，却又想知道它是什么形状。
- 做法是向山扔球/石头：
 1. 有些方向石头穿过去；
 2. 有些方向石头弹回来，并能听到声音；

3. 根据哪些方向穿过、哪些方向弹回，像 **ray tracing**（转写为 **retracing**）一样，反推山的形状。

- 讲者说这个比方让他非常震撼；小朋友大概听懂了，当然不可能百分之百听懂。
- 他认为震撼点有两层：一是模型有这个知识；二是他后来专门查资料，确认这个比方打得对。

GPT-5.6 因 reasoning 给出更严谨的解释

- 讲者接着看 **GPT-5.6**（转写为 GP5.6/GPT5.64号）：它给出了更严谨的答案。
- 讲者认为，因为它有 **reasoning**，所以还给了更好的解释，能更好地解释 GPT-4 时代的说法。

从 GPT-4 时代到 GPT-5.6 时代：同一工作从复杂生成缩成一句话

- 在 GPT-4 时代，讲者用 GPT-4 这个模型去生成奇怪的东西；一个 prompt 可以给 **GCLVM**（按转写）找到 150 个吧。
- 到今天 GPT-5.6 时代，这个工作“完全就是一句话的事情”，真的是一句话，不需要再为 GPT 做任何特别调整。
- 讲者的对比：GPT-4 时代：一个 prompt 生成/找到 150 个 → GPT-5.6 时代：一句话完成，无需特别调整。

Agent 的出现：长上下文 + 推理 + coding/ReAct 让语言模型能自主循环

- GPT 原本是一个能预测下一个 token 概率分布的模型。
- 随着大家发现模型可以输出越来越长的文本、接受越来越长的上下文，**agent** 就出现了。
- 讲者列出相关进展：有了推理模型，有了 **coding**，有了 **ReAct 式 agent**。

Agent 的机制：特殊 token 调用 bash → 结果回送 → loop

1. 语言模型（原模型）可以输出下一个 token；
 2. 也可以让它输出一个特殊 token，表示“调用一个 **bash**（转写为 **batch**）程序”；
 3. **bash** 程序运行；
 4. 运行结果重新送回到 agent；
 5. 这个过程在 loop 里重复。
- 结果：agent 可以自主运行；给它一个较复杂的任务，它可以不断尝试、尝试、尝试。

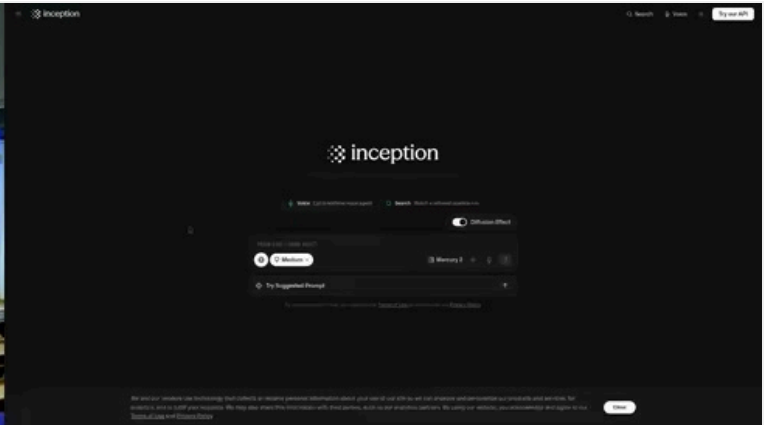
课堂展示配置：为看清屏幕而定制窗口管理器、top 与终端命令

- 讲者配置了一个 **space/SP 窗口管理器**（按转写）来优化课堂展示：对比度较高，顶栏等全部去掉，这样上课时能看得更清楚。
- 定制 **top**：默认不定制的 **top** 比较“土”；他的 **top** 更 fancy，能看到 OBS 是否还在运行、是否在浪费算力，以及内存还剩多少等。
- 终端配置得相对现代：
 - 敲 **ls** 时自动替换为 **eza**（转写为 EZA）；
 - 敲 **find** 时自动替换为 **fd**；
 - **fd** 的行为与 **find** 稍有不同。

第 9 部分



```
$ btop
$ eza --git --group-directories-first --icons=auto
docs  ▸ lect1  ▸ scripts  ▸ tentative  ▸ AGENTS.md  ▸ Makefile  ▸ README.md
$ eza --git --group-directories-first --icons=auto
docs  ▸ lect1  ▸ scripts  ▸ tentative  ▸ AGENTS.md  ▸ Makefile  ▸ README.md
$
```





Agent 已能把终端环境改成“现代、好用”的样子：一个 prompt 就让它动手改配置

- 讲者展示 `eza` 命令；当他要 `find` 时，也被自动换成了 `fd`。`fd` 的行为和 `find` 略有不同。
- 在 `YLS`（音；按上下文应是 `eza / ls` 的输出）里，输出很 fancy：目录、Markdown、Makefile 等都有特殊显示效果。
- 讲者说这些配置全是 agent 配的。他只给了一个 prompt：“你能不能帮我把我现在的命令行终端、把我的系统/思维都配置成现代、好用的样子。”
- 所用模型他提到 DeepSeek（原转写 deep sick）with flash（具体型号不清），并说“没有更好的模型”。
- （整理者串联）这个例子说明：ReAct agent 出现后，agent 不只是回答，还能进到 configuration（原转写 conversion）里反复改，最终把终端环境改好。

ReAct agent 变可用：错了会换方法，而不是直接停下

- 讲者描述 ReAct 类 agent：它不停在 configuration 里改，错了以后会试别的方法，最后把任务完成。
- 对比时间线：
 1. 去年讲操作系统课时，agent 还不怎么能用：各种出错，任务完成不了就停下来。
 2. 今年讲操作系统课时，agent 已经非常非常能用。
 3. 到现在，agent 产品爆发，到处是 work body（音）、豆包相关 agent 等广告。
- 讲者认为这就是 ReAct 类型的 agent。他觉得这个话题有点岔开，但“故事远远没有结束”。
- （整理者串联）这条时间线支撑的结论是：agent 的能力在短时间内从“不可用”跃迁到“可用并产品化”，但还没到终点。

Agent 能力还远未释放：next-token prediction 可能效率低

- 讲者认为 **next-token prediction** 是一个效率很低的模型。
- 他举 **diffusion model** 为例：让它写类似 `AI is love` 的内容时，生成过程从随机噪声开始，像 diffusion 图片一样一点一点生成很长文本。
- 这个过程像人做梦：梦里一个东西逐渐看清楚，越来越清楚。
- 他还提到未来可能性：
 - 不一定只 **predict next token**；
 - 可以回头改上下文；
 - 可以随时把很长的上下文裁短。
- 这些是今天很多人在研究的东西。
- （整理者串联）讲者用 diffusion 语言模型等方向说明：当前主流范式之外还有别的可能，所以 agent 能力未必只由 next-token 路线决定。

模型继续突破不可预测，但世界会变得很不一样

- 现在模型已经非常强；如果三个月后又突破、模型更强，一年后又突破、更强，整个 AI research 都在赌这件事发生，摩尔定律还在。
- 讲者明确说：他不能预测未来哪一条路线会胜出，哪一种模型会胜出。
- 但可以预见：这个世界肯定会变得非常不一样。
- 因此，每个同学都值得好好考虑“在这个世界里应该怎么办”。
- （整理者串联）讲者把技术不确定性和个人选择连起来：既然路线不可预测，重点就不是押注单一模型，而是准备应对变化。

吓人的部分说完后，转入“普通人如何实际操作 agent”

- 讲者说“吓人的事情说完了”，可以松一口气。
- 接下来要真的告诉大家应该怎么样实际操作 agent；这是这门课主要要讲的内容。
- 他先站在普通人的视角给大家科普。

小结：本段从 agent 改终端配置的实际案例，推到 ReAct agent 的可用性跃迁，再引出 next-token 范式局限与模型继续突破的不可预测，最后收束到“普通人如何操作 agent”的科普入口。

第 10 部分



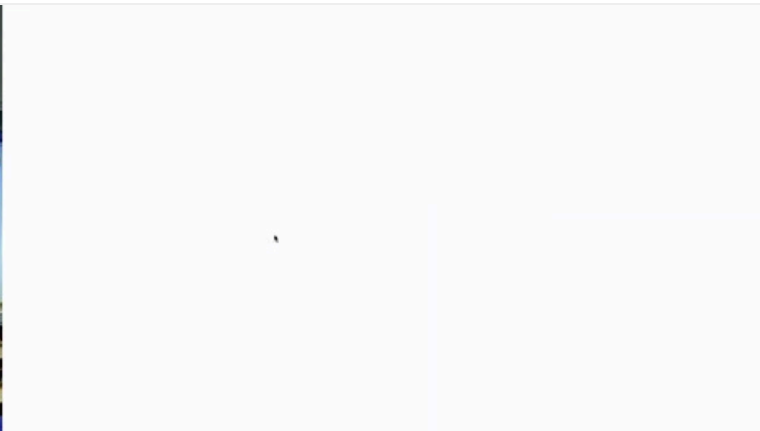
使用 Agents：普通人视角

To exit full screen, press and hold [Esc]

从“回答问题”到“把事情推进”

AI-native 的一代人，你真的应该用过。

- 课堂 demo：一个最小的 `llm`
- DeepSeek harness 会在某节课专门讲（首批内测，于是有了一个梗图）
- ✦ 最小跃迁：AI 不只给出答案，还会观察结果、采取动作、继续推进。



zhuanlan.zhihu.com

search

☐ Access other apps and services on this device

Block Allow

知乎日报

知乎日报



实际操作 agent：先假设不会用，再解锁用法

- 讲者说，如何实际操作 agent 是这门课主要要讲的。他先站在普通人视角科普，假设大家其实不会用 agent、不太会用 AI Agent；除了“帮我把作业做了”之外，使用方式可能需要“解锁”。
- 他以上学期期末题为例：题目最后考的是“如果你要用 agent 给我的课出期末卷子，应该怎么办”。由此他发现大家用 agent 的方式需要稍微解锁。
- 他欢迎同学有好想法随时打断、一起探讨。

使用 agent 的最小原则：开始用，随时间

- 作为 AI native 的一代人，要知道给 agent 一个合适的指令。讲者说课上选的是“派”[音]（具体工具名不确定）；你只要跟它说就可以了。
- 还可以写配置文件，例如 `AGENTS.md`。
- 演示对话：
 - 问它“你是谁”；它回答自己是“生成式软件工程的教学辅助围城”（疑为“助手”）。
 - 再问“你不就是个模型吗？为什么还知道生成式软件工程？”它回答：之所以知道这门课，是因为这个仓库里 `AGENTS.md` 的配置。

- 讲者总结原则：
 1. 第一个原则：开始，现在就开始用；
 2. 第二个原则：随时问他。
- 他把配置文件提供的上下文说成“临时的基因”。
- **（整理者串联）** 合适指令 + AGENTS.md 等配置 → agent 获得课程/仓库上下文 → 它能解释自己为什么知道这门课。

想象力是 agent 的上限

- 讲者认为：在这个意义上，是你的想象力限制了你的 agent 的上限。
- 他让同学课间想一想：你用 agent 做过的最酷的一件事是什么？并强调“想象力是你们的上限”。

实例：一句话让 AI 把 Debian 装到 U 盘，并用模拟器验证

- 背景：讲者今天上课的系统，所有配置安装都由 AI 完成。他带一个 U 盘，里面装 Linux 操作系统，可以直接插在教室电脑上启动；这是他一直以来的操作系统课特色。
- 轶事：有一年操作系统课上，他把教室电脑的硬盘抹掉了，结果下一位上课老师“教学事故”。这被用来说明/展示操作系统原理，也说明装机很痛苦。
- 讲者过去是“装机仙人”，有知乎链接；2021 年 7 月还是 pre-LLM 时代（转写为 PRLM/prim m），他各种定制系统。现在“装机仙人”已经被 AI 取代。
- 他的指令只有一句话：让 AI 去 `mirrors.nju.edu.cn` 找一个 `deb13` 镜像，把它装到 U 盘上；当时 U 盘插着；确保 U 盘可以启动；然后用模拟器验证。
- 讲者称点睛之笔是“用模拟器验证”。
- 操作链（讲者描述）：找 `deb13` 镜像 → 装到 U 盘 → 确保可启动 → 用模拟器验证 → 在 QEMU 中启动并看到登录界面。
- 他补充：他自己有 knowledge，所以知道 AI 是怎么完成这次安装的。

小结：讲者把 agent 使用落到最小动作——开始用、随时问；再用 U 盘 Linux 安装说明，上限取决于使用者能想到什么。

第 11 部分



Example 1: 现在运行的系统

Debian on USB

- 安装完全由 AI 完成，并且在 **QEMU** 中运行验证
- 不会用？直接生成文档
- 终端、nvim 的现代配置
- 我没有看过任何一眼配置文件，全部由自然语言完成配置
- 蒟蒻仙人彻底成为历史
- 彻底颠覆了计算机中“学习曲线太陡，不想用”的瓶颈
- 谁掌握了 AI，谁就掌握了暴增的学习效率，**先用好，再理解原理**
- ✦ 先用好 ≠ 不验收：学习曲线从“记住每一步”变成“判断结果是否正确”。



分两层回答：

1. 我说的“知道”有两层含义

- 训练时学的：我的参数里编码了大量文本语料中的统计规律，“生成式软件工程”这个词（LLM 辅助编程、AI 生成代码这些概念）在我的训练数据里反复出现，所以我“懂”这个概念，就像我“懂”操作系统、C 语言一样——这些是语言模型训练阶段习得的背景知识。
- 现在看到的：我之所以知道这门课，这个仓库，不是训练数据里有的，而是每次对话开始时注入的上下文——包括课程介绍、你的 AGENTS.md 配置，这部分是“临时记忆”，对话结束就没了。

2. 严格说，我没有“记忆”，只有“相关性”

我不像人一样存在一个知识库。我的行为是：根据输入，预测下一个 token 最合理的序列。你说“生成式软件工程”，我预测出与之最相关的表述——因为语料里这两者频繁共现，这更像搜索引擎的倒排索引，而非大脑的记忆。

所以准确的说法是：我不是“知道”，我是“大概率猜得上话”。不过对教学辅助来说，够用了一夜正我每次都猜得挺准（笑）。

Recap: 模型的“知识”= 训练语料中的统计规律（背景知识）+ 对话上下文注入（临时记忆），本质是 token 预测而非真正的记忆。

~/Projects/GenSE (main)
*107 4749 R6.5K C96.7% \$0.000 0.35/1.0M (auto) deepseek-v4-f

Agent 的安装能交付到可登录、可建用户、可验证

- 讲者先强调：他自己有 knowledge，所以能判断 agent 是怎么完成安装的。
- 具体安装链：agent 装完 → 在 **QEMU**（转写原文为“q email”，按上下文疑为 QEMU）里启动 → 看到登录界面 → 确认可以登录，并报告登录成功。
- 接着讲者要创建用户，指定用户名、密码；agent 全部做好并验证完成，再交给他。
- （整理者串联）这里的关键不只是“会装”，而是 agent 把启动、登录、建用户、验证串成了一条可验收的交付链。

Tiling window manager 也能靠自然语言配置，用户只负责验收

- 讲者启动 USB 镜像后，要求安装 **tile-based window manager**（平铺式窗口管理器）。
- 演示效果：按 **Windows/Super+Enter** 会产生新窗口，出现在右边，和常见窗口方式不同；按 **Windows/Super+空格** 可以把窗口挪出来。
- 不会用怎么办？他直接让 **DeepSeek V4 Flash**（转写为“deep sk v4flash”）根据当前系统设置生成文档。

- 文档告诉他：**Super+Enter** 打开 **terminal**，**Super+D** 打开 **app launcher**；例如 Windows + D 可打开 OBS 或 Chrome browser。
- 讲者称，所有文档、看到的终端、**Neovim**（转写为“new wave”）现代配置，他都没有看过任何一眼配置文件；一切由自然语言完成。
- 他只让 agent 保交付：提出需求，让 agent 再检查一下，就全部完成。
- （整理者串联）因此这里的工作流变成：人提目标、验收、追问；agent 改配置、解释、再检查。

学习方式变了：从读遍 CPU 手册，到判断 AI 的能力边界

- 讲者回忆：自己是教 **Computer Science** 的老师；大四时，可能和听众一样，看 Intel 几千页的 **CPU specification**，研究每一个 CPU feature 干什么用。
- 他现在的感觉：这个时间点有一个人/agent 可以和他沟通，已经把“这些东西”全看了、都懂、能做对；他需要的是**解锁它的能力**。
- 由此他说：今天学习知识的方式变了。你更需要知道 AI 能做什么，感知它能做到什么程度，然后带着它去做一件更大的、它做不到的事情。
- （整理者串联）这条链把“学习”从记住知识，推向判断能力边界、组织任务和验收结果。

讲者观点：人的唯一功能是带 AI 做它做不到的事

- 讲者明确说：“人现在唯一的功能就是带着 AI 做 AI 现在做不到的事情。”
- 进一步推论：如果没有这个能力，就立即已经被淘汰；如果没有 token，就可以立即退休。
- 他标明这是“我自己的观点”。
- （整理者串联）前面的安装、配置案例，被他用来支撑这个判断：人不必先亲自掌握所有细节，而要能引导 agent 完成交付。

Agent 打破 CS 工具“学习曲线太久、不想用”的瓶颈

- 讲者说，agent 彻底颠覆了 Computer Science 里学习曲线太久、令人不想用的瓶颈。
- 例子：大家都配置过漂亮终端，B 站上有人用花里胡哨的终端，你也想搞一个；但一改就出错，改配置文件 **.zshrc** 一改就出错，非常痛苦。
- 有 agent 后不痛苦：它不仅能改，还能事无巨细解释为什么要这样改；有新想法一提，它就说好好好，还能提供情绪价值。

- 讲者前几天在小红书发帖：为什么 token 比学生好用？因为可以骂它，它不会还嘴；而他肯定不能骂学生。这是讲者的玩笑式个人表达。

第 12 部分



Example 1: 现在运行的系统

Debian on USB

- 安装完全由 AI 完成，并且在 QEMU 中运行验证
- 不会用？直接生成文档
- 终端、nvim 的现代配置
- 我没有看过任何一眼配置文件，全部由自然语言完成配置
- 装机仙人彻底成为历史
- 彻底颠覆了计算机中“学习曲线太陡，不想用”的瓶颈
- 谁掌握了 AI，谁就掌握了暴增的学习效率，**先用好、再理解原理**
- ♦ 先用好可不验收：学习曲线从“记住每一步”变成“判断结果是否正确”。

token/agent 比学生好用的关键：可以骂，且不还嘴

- 讲者的玩笑式判断：为什么 token 比学生好用？因为可以骂它，它不会还嘴。
- 对比：讲者认为自己肯定不能骂学生，但可以骂 agent。
- 讲者由此提出：谁掌握了 AI agent，谁就掌握了暴增的学习效率。
- 讲者推荐的学习方式：先用好它，先把东西搞出来，再回头复盘理解。这比从零开始一点一点搭积木效率高很多。
- 但讲者指出危险：这样做时，你的理解可能与“比较好的理解”有差距；门道我们不知道，但这是非常不一样的事情。

邮件场景：个人 skill 自动读信、写草稿、归档附件

- 讲者说第二个例子是收发邮件、写草稿。
- 他有一个自己的 skill（不展示）。他的数据库里有历史上回过的所有邮件。
- 流程：
 1. 收到邮件 → 自动变成已读状态；
 2. 自动写好回复草稿，用讲者的口吻；
 3. 重要附件（例如发票）自动保存到指定归档位置。

- 这些行为都来自他写好的 **instructions**。
- 讲者对效果的评价：**目前还不够好**。简单 routine 性质的东西可以直接用它的回复；大部分时候还得改。
- 但他因此得到一个 **AI bot**。

用 Apple Reminders 当个人 agent 入口：待办即指令，后台执行后推送

- 讲者说，他可以像用 OpenClaw（原文作 open cloud / open cow）或 ARMUS（原文）一样与自己的 agent 交互，但他用的是自己的 **Apple Reminders**。
- 操作方式：在手机提醒事项/备忘录里加任务，例如“帮我把今天的发票打印出来”。即使只是浪费一张打印机的纸，他现在也能做。
- Demo 细节：在一个叫 **bot** 的列表里（当时是空的），加入“把我收到的上一个发票打印出来”；点勾就 **commit**，多了一个 to-do 事项。这跟在 Telegram 里跟 bot 聊天一样。
- 之后只需等：agent 在后台处理，完成后重新推送通知；通知通过 **Reminders** 送到手表，手表告诉他“可以打印了”。
- 讲者结论：所有日常事务都可以交给它处理，前提是**自己放心的事情**。
- 讲者认为：这比 Telegram 聊天框要好；而且这个流程会不断进步。他在 OpenClaw 之前就这样做了。
- 对个人 agent 产品的判断：讲者疑问/思考 **OpenClaw 或 ARMUS 是不是个人 agent 产品的终极形态**；他认为这个产品还可以做得更好，未来还有很多空间。

阅读与论文：讲者自己的阅读状态已经改变

- 讲者问听众：你们还在读书吗？还看书吗？还看论文吗？看论文痛苦吗？现场有人说痛苦/不痛苦，大家在笑。
 - 讲者自述：他是一个很喜欢看书的人；在听众这么大的时候，每年教科书就要看几十本，还会看很多闲书。
 - 但今天他已经不看书了。
 - 例外/现状：他还是买了一本，因为要上“**生成式软件工程**”课，所以又买了一些。
-

第 13 部分



Example 3: 读书、读论文

从古法学习到电子版 Agent

- 我又买了以前看过的书，重温一下古法学习
- 随后立即电子版 AI Agent 启动 🤖
- Extension: 读 b 站，生成的讲义
- ✦ 生成讲义只是转换材料，不自动等于“深度阅读”。

AI 如何辅助深度阅读？

- ✦ 追踪论证：结论是什么？证据在哪里？每个回答都能跳回原文。
- ✦ 对抗阅读：让 Agent 提出反例、遗漏的前提，以及作者最强的反驳。
- ✦ 主动回忆：合上原文接受口试，再根据原文纠错：摘要不能替你理解。
- ✦ 重新组织：把新观点改写、归类，并连接到自己的旧笔记与真实问题。



```
- A snippet (usually a part of the file).
Goal: create a slideshow of ONLY THE GIVEN SNIPPET (not the full file content). Only produce markdown slides fragments.
No title/ending pages.

## Instructions
- First, read the full lecture notes. (But the full contents are FV1; only need to generate for the given snippet).
- My snippets are my logic line; and you can extend my snippet (or give comments) further with thoughtful, non-trivial adds.
For each slide, fan out three subagents (give them sufficient context) of expert:
1. Thinking of historical recaps, lessons, accidents, etc.
2. Search for facts or related work, in the human history, not merely computer science. Prefer well-recognized, high-impact results.
3. Think in first principle.

After collecting the three responses, you can finalize the content: No trivial summaries and derivations--I knew that and can talk in the class. Keep concise, thoughtful, and insightful. Only converge to really useful add-ups.
- Always expand "[[...]]" enclosed parts: this is asking for AIQC.
- Focus on the snippet itself. You can extend but not dismiss my ideas. Never reference later contents passing this snippet's point.
- Keep all referenced links as-is.
- No generated diagrams unless instructed. For generated raster images, embed base64 webp, downscale to 720p. Generate picture in subagent, prefer diagrams with information (Chinese characters), white background, prefer the most powerful model.
- Save markdown slides to '/tmp/slides.md' and end immediately: you run in headless. No output.

Style:
- You can add lists/h2/etc; but surround all your added text by '<span class="aigc">'. Minor revised text should not be
agents/skills/make-slides/SKILL.md
-- VISUAL LINE --
```

为了教“生成式软件工程”，重新带回读书时反复读过的经典

- 讲者说自己现在已经不看书了，但因为要上生成式软件工程课，又买了一些、带了几本书。
- 这些书是他在读书时仔细看过、甚至看过不止一遍的书。

《人月神话》：标题误导，“神话”其实是“鬼话”

- Fred Brooks, The Mythical Man-Month. 讲者认为这个标题起得非常 misleading。
- “人月”不是“人和月亮”，也不是月亮出来变成狼人的“人狼”；这里对应的是 man-month，是人力/工作量单位。
- “神话”也不是盘古开天地那种美好、宏大的神话。讲者认为这本书按意思更像是“人月鬼话”。

《软件设计哲学》：紧凑，密集的软件工程智慧

- 讲者还读了 John Ousterhout, A Philosophy of Software Design (录音作“john nasra / the philosopher of software design”)。
- 他认为这本书非常紧凑，感觉比当年看的英文版更紧凑；里面有大量真正的软件工程智慧。
- 他重温了当年读书的感觉，读了第一章。

备课时让 GPT-5.6 先查历史案例，再找跨领域高影响相关工作

为做每一页 slides，讲者说会让 GPT-5.6：

1. 先回看人类历史中与主题相关的经验教训/重要案例，看有没有能得到启发和印证的东西。
 2. 再根据讲的内容找相关工作，不只在 **computer science**，并偏好 **very recognized / high impact** 的资源。
- 原因：这是上课，讲者希望讲经典的东西。
 - 讲者认为在这部分 skill 里，自己的做法比 GPT-5.6 更好一点；他的 **AI agent 同学**（录音作“AI 梗同学”）会更强、更 **harsh** 一些。

多 agent 抽卡、分歧、回送讨论，直到收敛：算力换智力

- 讲者在 AI agent 里加入更多“**first-principles thinking**”人格。
- 加入随机性抽卡。
- 抽完后，agents 讨论，会产生分歧观点。
- 主 agent 再把分歧观点送回给他们继续讨论，直到**收敛**为止。
- 讲者把这条路称为“**算力换智力**”。这样他可以去审稿论文。

读论文/review：先假设已有知识已知，只剥离最少新增量

- 讲者的读法/review 法：
 1. 从人类已有知识开始。

2. 假设人类已有的所有东西都已经固定下来、都是已知的（讲者纠正：不能说都是 trivial 的，但都是已知的）。
 3. 把论文里新的、最少的东西剥离出来。
 4. 这时一篇 12 页论文基本就没剩下多少，会变得非常非常短。
- 因此他现在读论文效率非常高。
 - 他还会和 AI chat：拿论文原文、总结、深层次总结一起聊。
 - 结果：读一篇论文的效率比以前高得多。
 - 讲者强调：他仍然读论文，不是那种只看微信公众号的“老登”。

第 14 部分



Example 3: 读书、读论文

从古法学习到电子版 Agent

- 我又买了以前看过的书，重温一下古法学习
- 随后立即电子版 AI Agent 启动 😊
- Extension: 读 b 站，生成的进义
- ✦ 生成讲义只是转换材料，不自动等于“深度阅读”。

AI 如何辅助深度阅读？

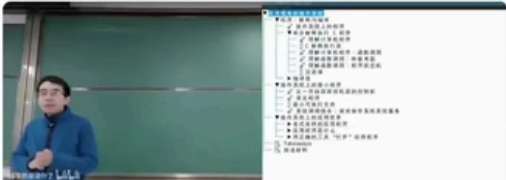
- ✦ 追踪论证：结论是什么？证据在哪里？每个回答都能跳回原文。
- ✦ 对抗阅读：让 Agent 提出反例、漏洞的前提，以及作者最强的反驳。
- ✦ 主动回忆：合上原文接受口试，再根据原文纠错；摘要不能替你理解。
- ✦ 重新组织：把新观点改写、归类，并连接到自己的旧笔记与真实问题。





02 - 应用视角的操作系统 [2026 南京大学操作系统原理]

1:27:56 · BV1KNPh4PErE

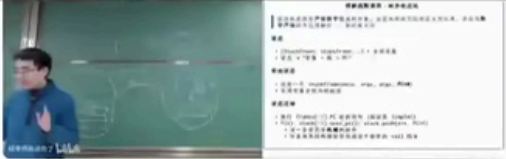


今天我们来讲应用视角的操作系统。上一节课我们很快地回顾了操作系统发展的历史，我相信在一定程度上已经说服了你们——即便是在人工智能的时代，我们还是应该坐下来好好学操作系统。今天我会从这个学期操作系统最重要的视角出发，也就是应用程序认为操作系统就是一组 API，来理解操作系统。今天我要用一节课的时间，把应用视角的操作系统到底是怎么看、怎么理解的，做一个 overview。所以今天的主角就是程序。我讲的第一部分是：解释和编译一个程序，也就是操作系统上的程序。先回顾一下操作系统的定义：Operating systems are a body of software, in fact, that is responsible for making it easy to run programs. 它可以让程序共享资源，可以让程序与设备交互，还有各种各样有意思的事情。也就是说，操作系统给你提供了 API，赋予你非常大的能力，让你能去做以前在 C 的控制台里只用 printf 做不到的事情。如果你了解了操作系统的这些 API，就意味着你可以用操作系统干更多的事。所以我觉得学一学还是值得的，尤其是当你知道操作系统可以做什么以后，你就可以指挥 AI 去调用这些 API，把它真正地做起来。大家都知道人工智能很强，但如果你没有合适的 prompt，它就在那里什么也不动。我们会在操作系统课上——或者今天——彻底讲清楚什么是程序，精确到每一个程序的 bit 都是 well-defined 的。顺便说一句，今天你们算是第一次接触“计算机程序的形式语义”这个概念，我不会专门讲这个概念，但你们会感受到什么是编程语言的形式语义。好，那我们就开始吧。



史，我相信在一定程度上已经说服了你们——即便是在人工智能的时代，我们还是应该坐下来好好学操作系统。今天我会从这个学期操作系统最重要的视角出发，也就是应用程序认为操作系统就是一组 API，来理解操作系统。今天我要用一节课的时间，把应用视角的操作系统到底是怎么看、怎么理解的，做一个 overview。所以今天的主角就是程序。我讲的第一部分是：解释和编译一个程序，也就是操作系统上的程序。先回顾一下操作系统的定义：Operating systems are a body of software, in fact, that is responsible for making it easy to run programs. 它可以让程序共享资源，可以让程序与设备交互，还有各种各样有意思的事情。也就是说，操作系统给你提供了 API，赋予你非常大的能力，让你能去做以前在 C 的控制台里只用 printf 做不到的事情。如果你了解了操作系统的这些 API，就意味着你可以用操作系统干更多的事。所以我觉得学一学还是值得的，尤其是当你知道操作系统可以做什么以后，你就可以指挥 AI 去调用这些 API，把它真正地做起来。大家都知道人工智能很强，但如果你没有合适的 prompt，它就在那里什么也不动。我们会在操作系统课上——或者今天——彻底讲清楚什么是程序，精确到每一个程序的 bit 都是 well-defined 的。顺便说一句，今天你们算是第一次接触“计算机程序的形式语义”这个概念，我不会专门讲这个概念，但你们会感受到什么是编程语言的形式语义。好，那我们就开始吧。





你们可能需要回去再花一点时间看一下。如果你们看到这样一个非递归汉诺塔的时候，你们可以去调一下这个程序，你就真正知道我所说的：所有的计算机世界里你看到的东西都是没有魔法的。计算机世界没有任何魔法，它都是数学，都是你用离散数学可以定义出来的东西。什么叫 call？你看，所谓的 call 就是：我往这个栈上面 push 一个 frame，frame 的 PC 是 0，PC 是 0，我 push 什么值就是什么值，然后我的 return 就是 pop，pop 一个值。然后我的 C 程序，比如我要做一个非递归的汉诺塔：我就先 call，我要把 N 个盘子从 A 经过 B 到 C；然后我这个 call 注意调用的是我写的这个函数，这个函数就是把一个帧栈压上去。然后接下来我的程序的模拟器——相当于我的 C 的模拟器——就开始执行了。C 的模拟器就从这样的一个状态上，机械地遵循我所说的：什么是在执行一条 C 语句？从 PC 上面取一条语句出来，跟



```
例子：为什么不是 Pr(context_{t+1} | context_{t}) 的模型呢？
- [Diffusion_LM](https://chat.inceptionlabs.ai/)
- Self-modifying of context...

## 使用 Agents: 普通人视角

AI-native 的一代人，你真的应该用过。课堂 demo: 一个最小的 [Pi](https://pi.dev), DeepSeek harness 会在某节课专门讲 (首批内测，于是有了一个模因)



你的**想象力**限制了 Agent 能完成的任务

Example: 现在运行的系统

- Debian on USB: 安装安全由 AI 完成，并且在 QEMU 中运行验证
  - 不会用？直接[生成文档](../docs/sway-cheatsheet.html)
  lect1/index.md 71,0-1 27%
```

```
/home/jyy/Projects/GenSE/scripts:
buildall
__pycache__
slides.html
Took 0.0s

Thinking...

$ cd /home/jyy/Projects/GenSE 66 head -40 scripts/slideshow
66 echo "" Makefile "" 66 cat Makefile
... (64 earlier lines, ctrl+o to expand)
dry-run:
$(PYTHON) $(BUILDALL) --dry-run

build:
$(PYTHON) $(BUILDALL)

Took 0.0s

Thinking...

这是你的 2026 春季《生成式软件工程》课程仓库
(/home/jyy/Projects/GenSE)，工作区干净，最近十几次提交全
```

用 AI chat 提高读论文效率，但仍亲自读论文

- 讲者说跟 AI chat 之后，现在读一篇论文的效率比以前高得多。
- 他仍是读论文的；并先 claim：自己不是那种只看微信公众号的“老登”，不是靠微信公众号学习的人。
- （整理者串联）这里他在划边界：AI 提高的是效率，不是把“亲自读论文”这个环节取消。

AI/Agent 已能接管电脑操作：打印发票是具体例子

- 讲者收到通知“发票已打印”，只需回去到打印机拿走。
- 因此它“可以干任何事，literally”：那台电脑能做的事情，所有都能干。
- 他设想如果有时间，或者大学倒闭了，就弄一个机械手让它做更多事情；现在做不到，但它已经能操纵打印机。
- （整理者串联）这个例子说明 agent 的能力不只在对话，而在操作真实电脑任务；这为后面“用 agent 学习”提供了行动层面的基础。

也用 AI 上 B 站；娱乐内容别外包

- 讲者不仅用 AI 读书、读论文，还用 AI 上 B 站。
- 但如果只是娱乐，比如看“华强买瓜”的衍生视频，那就不要用 AI 代看/代读；否则会剥夺娱乐。
- （整理者串联）他在这里区分：用于学习/信息获取，和用于娱乐，不是同一种外包。

长视频学习流水线：BVID → 图文时间轴 → 截图 ground truth → AI 过滤

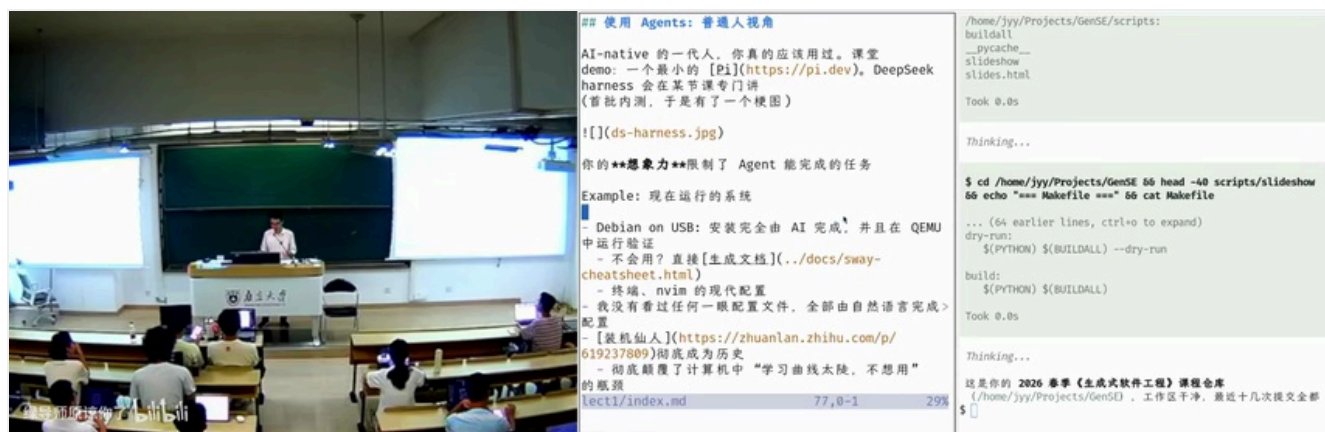
- 讲者会给 AI 一个 BVID，例如一个 1 小时 27 分钟的视频/播客。
- 他举例提到一个播客/UP主（转写为 **Macbook cup**）发的“软件工程实践”相关内容；讲者认为这个人挺“神棍”，有些经验不完全适合他们的课程。
- 流程是：
 1. 把视频自动总结成**图文时间轴**。
 2. 整理过的文本质量还可以，比字幕质量更高。

3. 图文版图片稍少；实际截图密度更大：当两帧图片之间的间隔大到一定程度，就截图，得到视频所说的 **ground truth**。
4. 这时 **AI耿同学** 上线，替讲者消化一个多小时的视频。
5. 因为 AI 有**我的记忆**，它能去掉讲者认为 **trivial**、已经非常清楚的东西。
6. 于是讲者可以在几分钟内看完 1—2 小时的视频，并基本确信自己知道了里面“不一样的东西”。

AI agent 时代：学习效率差距 10—100 倍，竞争问题回到简历

- 讲者认为这是非常可怕的事情。
- 在 AI agent 时代，会用 agent 学习的人与不会用 agent 学习的人，学习效率差距是 10 倍到 100 倍。
- 他追问听众：坐在这里，怎么和一个效率高 10/100 倍的人竞争？
- 又回到开头的问题：如果所有人的简历都一样，那么那个没被看到的、学习效率是你 100 倍的人，他的简历会长什么样？
- 讲者说这是非常可怕的事情；更可怕的还在下一节课。课间休息，让听众想一想；下节课再讲比刚才更可怕的，并留一点惊喜。
- 结尾提到“所以为什么我要说这个大蛇丸啊”，作为下一节悬念的引子。

第 15 部分



```
## 使用 Agents: 普通人视角

AI-native 的一代人，你真的应该用过。课堂
demo: 一个最小的 [Pi](https://pi.dev)。DeepSeek
harness 会在某节课专门讲
(首批内测，于是有了一个模版)



你的**想象力**限制了 Agent 能完成的任务

Example: 现在运行的系统

- Debian on USB: 安装完全由 AI 完成，并且在 QEMU
  中运行验证
  - 不会用？直接[生成文档](../docs/sway-
    cheatsheet.html)
  - 终端、nvim 的现代配置
  - 我没有看过任何一眼配置文件，全部由自然语言完成>
    配置
  - [装机仙人](https://zhuanlan.zhihu.com/p/
    619237809)彻底成为历史
    - 彻底颠覆了计算机中“学习曲线太陡，不想用”
      的瓶颈
lect1/index.md 77,0-1 29%
```

```
/home/jyy/Projects/GenSE/scripts:
buildall
__pycache__
slideshow
slides.html

Took 0.0s

Thinking...

$ cd /home/jyy/Projects/GenSE && head -40 scripts/slideshow
&& echo "=== Makefile ===" && cat Makefile

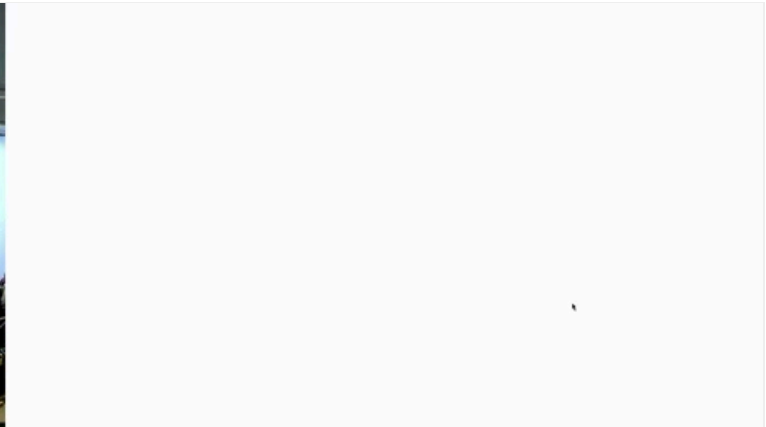
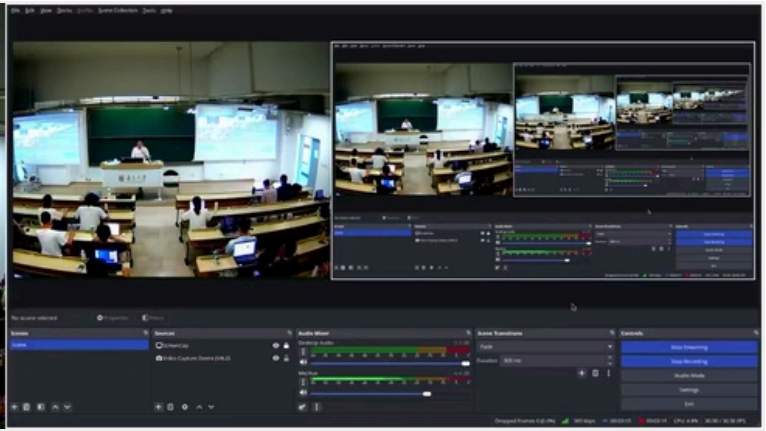
... (64 earlier lines, ctrl+o to expand)
dry-run:
$(PYTHON) $(BUILDALL) --dry-run

build:
$(PYTHON) $(BUILDALL)

Took 0.0s

Thinking...

这是你的 2026 春季《生成式软件工程》课程仓库
(/home/jyy/Projects/GenSE)，工作区干净，最近十几次提交全部
$
```



从白板 Agent 到完整生态 To exit full screen, press and hold [Esc]

白板 Agent 只是起点

- 注意：以上任务，**白板 Agent** 就能完成
- Agent 拥有完整的生态系统！
- domain-specific 任务 → domain-specific 插件
- 计算机科学定律：你想做的事，一定也有别人想做
 - ✦ Engelbart 早在 1962 年就指出：增强智能来自“人 + 工具 + 语言 + 方法”的系统，而不是一个孤立的聪明部件 (*Augmenting Human Intellect*)
 - ✦ Agent 能力 $\approx$ 推理 $\times$ 可观测性 $\times$ 可操作性 $\times$ 反馈

下节课会更可怕：讲者先承认自己受限，以此强化“外面有人做得更好”的判断

- 讲者预告：下一节课要讲“比刚才更可怕”的内容，并用“大蛇丸”“人类完了”承接前文判断。
- 他知道听众常觉得他像“神棍”，但他强调自己就是这样想的。
- 他的关键确信：世界上一定有人做得比他更好。
- 自述限制：他只是一个“上有老下有小”的人，没办法全力投入；上学期为操作系统课花了很多时间。如果不花这些时间，本可用来探索“这件事应该怎么搞好”。

- 逻辑链：讲者受家庭/教学约束 → 仍有此判断 → 世界上有人做得比他更好 → 他的“人类完了”不是他一个人的夸张（整理者串联）。

直播事故：USB 采集卡断连，Agent 用一个 prompt 用软件“拔插”恢复

- 场景：直播时 USB 摄像头卡了。教室里有采集卡，视频采集卡可能从网络或别处接入，还有好几台机器。讲者认为学校用了成本很高的方案；他觉得 500~1000 块可能就能解决整个问题。
- 故障：采集卡 USB 断开，原因不明。讲者现场说“左边这个我挥挥手，它是在动的”，但 USB 断了。
- 处理：讲者立即让 Agent 修，只给了一个 prompt：我的 USB video 有问题，能不能 reconnect。Agent 先查看系统 USB 设备情况，找出设备。
- 操作：用的是讲者“前所未见的 Linux 命令”；他有些概念，但不知道具体怎么工作。Agent 用软件方式把 USB 采集卡“拔出来，又插进去”。
- 为什么需要这样：接口在机器后面，讲者碰不到。
- 结果：Agent “拔插”后就好了；视频目前没问题。若再出问题，讲者请听众打断他，他再重置。
- 逻辑链：USB 断连 → 物理接口不可达 → 讲者发 prompt → Agent 查 USB 设备并用 Linux 命令软件重连 → 视频恢复。

这还不是 AI 真正形态：空白平台 + API 就能复现刚才能力

- 讲者说刚才展示的还不是 AI 的真正形态。
- 对普通人：从 **pad 官方网站（[[原文如此]]）**安装一个“白板的派（[[原文如此]]）”，什么也没有；配置一个 DeepSeek V4 Flash（[[原文如此]]）的 API，就能立即得到刚才所有那些东西。
- 讲者认为：这很可怕。

Agent 的真正扩展在生态：开源脚本与 skill 让能力复用变得极轻松

- 讲者强调：Agent 远不止于此，它能拥有一个完整的生态系统。
- 当有领域特定任务时，讲者引用他课上常说的 computer science 定律：如果一件事合理且你想做，那么 100% 一定也有别人会想做。
- 在 AI 时代，开源社区会公开一些脚本。讲者说自己今天又发布了新脚本到 GitHub，可以干这件事；你把它拉下来，就能做自己的事。
- 因此，在 AI 时代复用别人的能力变得前所未有的轻松。

- 他可以把 skill 发到 Skill Hub；你也可以在 pad 插件生态 ([原文如此]) 上面找到一个.....

小结：讲者用直播 USB 故障证明 Agent 已能接管系统级操作，又用“空白平台 + API”和开源/skill 生态说明这种能力会被普通人低成本复现。

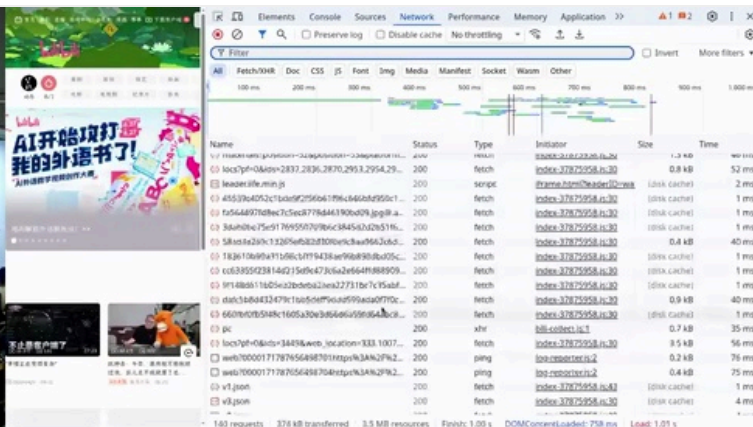
第 16 部分

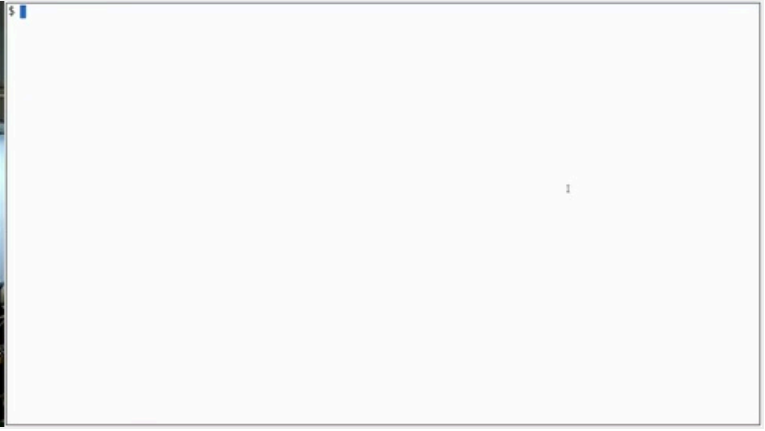


从白板 Agent 到完整生态

白板 Agent 只是起点

- 注意：以上任务，**白板 Agent** 就能完成
- Agent 拥有完整的生态系统！
- domain-specific 任务 → domain-specific 插件
- 计算机科学定律：你想做的事，一定也有别人想做
 - ✦ Engelbart 早在 1962 年就指出：增强智能来自“人 + 工具 + 语言 + 方法”的系统，而不是一个孤立的聪明部件 (Augmenting Human Intellect)
- ✦ Agent 能力 $\approx$ 推理 $\times$ 可观测性 $\times$ 可操作性 $\times$ 反馈





逻辑链：AI 降低能力复用成本，普通人下一步是管控浏览器和电脑

讲者在 AI 时代背景下提出一条路径：

1. 在 AI 时代，“复用别人的能力”变得前所未有的轻松。
2. 具体机制是：我往 Skill Hub 发一个 Skill，你可以在“pad 插件生态”（转录原词，名称不确定）里找到对应插件/能力。
3. 因此普通人的下一步应是：用插件把浏览器管控起来，用插件把电脑管控起来。

浏览器是软件工程奇迹，Chrome DevTools 是专为开发者设计的接口

- 讲者以他们用的 Chrome 为例，认为浏览器是很好的工程产品，可说是人类软件工程史上的奇迹之一。
- 它看起来只是显示网页，但非常复杂：代码量甚至远超操作系统内核；因为它要和用户打交道，涉及图形栈、显示相关、性能优化等。
- 浏览器有一个专为开发者设计的接口：Chrome DevTools。他提到大家应打开 Chromium/Chrome 按 F12。
- 可做的操作：改/删页面元素（讨厌某个东西就 delete 掉）、调试页面；在 Console 里网站常嵌入招聘广告——讲者提到 B 站，现场 F12 没看到，但他记得 B 站也有；还能看 HTTP 头 等信息。
- 还可以 copy 链接/命令，让它带着浏览器的 cookie 和所有状态，用 curl 去请求，从而得到列表。

LLM 时代的变化：以前开发者做的事，现在普通人也能用

讲者认为，上面这些都是以前 pre-LLM 时代 开发者做的事情；但到了 LLM 时代变了，普通人也可以享受到 Chrome DevTools 所有厉害的地方。

2020 年项目：把 Android app 搬到浏览器里

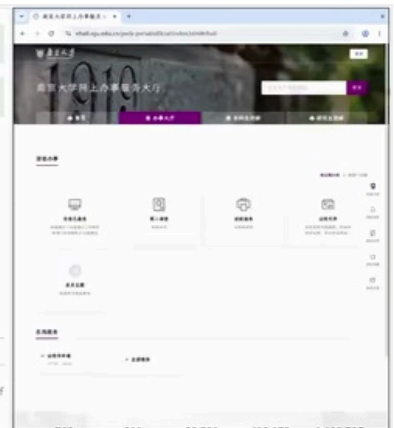
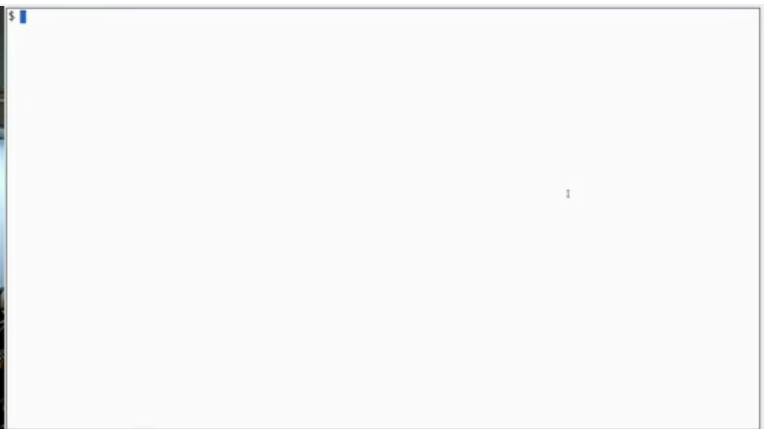
讲者回忆 2020 年做过一个非常疯狂的 project，并评价这类工具非常非常 powerful：

- Android 应用可以用 Java 写；Java 或 Kotlin 都编译到 **Java 虚拟机**。
- 于是可以在 **OpenJDK** 上运行一个 **Android app**。
- 运行 Android app 时，把界面上按钮的 **bounding box** 坐标、文本都算出来。
- 但不走 Android 那套渲染，而是直接在浏览器/Chrome 的一个框里显示一个**计算器**：每个按钮都在，字也对；点击后计算器状态会变化。
- 所以可以在本机执行一个 Android app。讲者明确说今天不讲这个。

转向教务系统：一个被讨厌的软件工程案例

讲者随后说，接下来要干一件事：**你们都讨厌教务系统吧，我也很讨厌**。他会在讲到软件工程时特别讲这一点，并说教务系统是一个软件工程的……（此处转录中断）。

第 17 部分





从空白表单到自动抢课：讲者的完整逻辑链

1. 教务系统要求空白教学周历 → 讲者不愿手动填。
2. 用能控制浏览器的 agent 输入自然语言指令，让它随便填 AI SLO（原文多处识别为 AI s law / AI s log / AI slap）。
3. agent 一开始不听指令 / 翻车 → 通过反思与试错，用 Chrome DevTools 发现直接注入事件不行，但鼠标点击可行。
4. 一次走通后，session 里的路径会留下来 → 可以沉淀成确定性的脚本和记忆。
5. 同一能力可以把走过的网页全部变成脚本 → 得到自动选课抢课系统。
6. 讲者结论：这很恐怖，不需要任何知识；这不是单个反例，而是一整个生态。

教务系统被讲者定位为软件工程的反面例子

- 讲者说“你们都讨厌教务系统吧”，他自己也很讨厌。
- 他预告：讲到软件工程时，会特别把教务系统当作一个非常反面的例子来讲。

演示：让 agent 自动填充空白教学周历

- 当前教学周历是空的。
- 讲者先在某 AI（“派”）里让它打开浏览器；这个浏览器属于“派”。
- 他讨厌动不动被要求填东西，所以日常会用豆包输入法：按一个键就和它对话。
- 但这里他的 Linux 没有豆包输入法，而且他好像没有配置语音输入，因为声音输入可能有点麻烦。
- 指令大意：现在教学周历是空的，帮我用随便什么 AI SLO 填起来；不要看任何东西。

- 不让它看的原因：如果 agent 知道当前目录是这门课的 **repo**（原文：ripple），它就会去读讲者的 **lecture notes**，这不是讲者想要的。
- 结果 agent 不听指令，仍然去干别的事。讲者把这次问题归因于 **DC with flash / deep with flash**（原文如此，疑为某个 Flash 模型组合）；他说如果用最一线模型，会遵循得更好。用 Flash 的另一个原因是它在课上更快。
- 讲者补充：模型能力还在不断增长中，翻车也是经常有的。

Agent 的关键能力：试错、反思、用 Chrome DevTools 找到可行路径

- 讲者说这就是 **agent**：你会看到它的过程。
- 它会不断反思自己做的事情合不合理，然后找到应该怎么办。
- 开头看起来像翻车，其实只是它刚开始尝试；尝试之后，它找到了正确方式。
- **Chrome DevTools** 是一个非常强大的工具：它最终发现直接注入事件的方式不行，但点鼠标是可以的。
- 这意味着它已经开始可以填写 **AI SLO**。
- 讲者完全可以不需要值守，让它随便填点什么。他的态度是：Yes, I love it. Doesn't matter.
- 他反问：能力真正的边界在哪里？

一次走通后，路径可沉淀为脚本和记忆

- 当 agent 走通过一次以后，这个 **session** 里面的东西会留下来。
- 它可以被沉淀成确定性的脚本。
- 它可以形成记忆：这条路径应该怎么走，什么能成功，什么能失败。
- 它也可以把走过的这些网页全部变成脚本。
- 于是你得到一个自动选课抢课的系统。
- 讲者认为这非常恐怖：你不需要有任何知识，它就在这里帮你填写 **AI SLO**。
- 如果本科生院一定要他填，他就用这个方式来对付。

这不是单个反例，而是一整个生态

- 讲者强调：这不是“好吧，又是一个反面案例”而已，而是一整个生态。

- 他说自己今天已经讲了好多类似案例。
 - 他的个人态度是：我可以这样用，这样用。
-

第 18 部分

AI Agent 的底层逻辑是 Unix 管道式组合：不是单点技巧，而是生态

- 讲者先收束前面展示的众多用法：每个都能“这样用、那样用”，但真正关键的不是又一个范型案例，而是一整个生态。
- 他用自己的操作系统教学背景解释：Unix philosophy 能成立，靠的是管道 (pipe)。管道让操作系统上的应用程序可以自由组合、无限复用。
- 因此不是 $(1+1=2)$ ，而是 $(1+1>2)$ 。
- 平移到 AI Agent：从一个浏览上下文中提取记忆和脚本的 skill、浏览器 skill、网页浏览 skill 等，可以加在一起、互相调用。
- 讲者结论：AI Agent 生态已经达到临界点；马上会看到这些能力互相组合，成百倍、成千倍放大个人能力。

逻辑链（讲者给出）：

1. Unix 管道 → 程序自由组合、无限复用 → $(1+1>2)$ 。
2. AI skill / 脚本也可互相调用 → Agent 生态达到临界点。
3. → 组合带来的个人能力放大不是线性，而是百倍、千倍级。

一般人的机会：把信息源沉淀成 skill / 脚本，搭起“知识飞轮”

- 讲者认为普通人可做的一件事：找到世界上各种各样的信息源。
- 例子：订阅橘鸦的 AI 早报（原音近“局丫”），每天早上刷牙时听一下；它由 AI 生成，但有 RSS / 推送源。
- 其他信息源：B 站推送、arXiv 推送（原音近 archive）、Hacker News 推送。
- 这些信息源都可以“做一次”沉淀成 skill 或脚本。
- 之后进入一个信息处理系统：
 - 大量信息源 → 处理系统 → 有用的知识。
 - 处理系统由各种各样的 skill 构成。
 - 这些 skill 都是自然语言，讲者强调“所有东西一行代码都不用写”，但仍可复用。

- 人会不断调整工作流；工作流不断产生对你有帮助的知识 → **飞轮转起来**。
- 讲者用对比强调：一个有这种飞轮的人 vs 没有飞轮的人，差距“多么可怕”。他把这种状态夸张为“拥有飞轮飞轮的飞轮”。

链条：信息源 → 沉淀为 skill / 脚本 → 进入处理系统 → 产出有用知识 → 调整工作流 → 飞轮。

现象级产品的惊喜：自我进化，但讲者对 AI 总结 memory 有保留

- 讲者由此解释为什么 Open Cloud 和 AMOS（原音保留，具体产品名不确定）是现象级产品、为什么能给人惊喜。
- 他提到的惊喜能力之一是**自我进化**（原音作“自我净化”）。
- 但作为 CS professional，讲者说他不惊讶，甚至讨厌让 AI 去总结 **memory**：
 - 因为 AI 有自己的提示词，会把不想记住的东西记住，把想记住的东西不记住。
 - 所以他更多时候调用自己的流程：把内容写到自己的数据库，再让 AI 去做这些事。
- 讲者进一步推测：现在产品还不够成熟、模型可能还不够强；一旦模型过了临界点，他现在只比 AI 多一点点；如果那一点点也被 AI 取代，他也会被消灭。他说“我也不强调”。

更激进的应用示例：录屏一天，让 AI 自动改进你的流程

- 讲者举出前面提到的录屏：录屏一天，让 AI 自动改进你的流程。
- 具体入口是问 AI：“我这一天干的这些事……”（本段到此为止，后续未展开）。

小结：本段核心是：AI Agent 的价值不在单个 skill，而在 Unix 管道式组合、复用和由此形成的个人知识飞轮；讲者对 memory 总结和自身被替代保持清醒，但不展开。

第 19 部分

普通人的视角：录屏一天，让 AI 找流程提升空间

- 讲者提出一个具体做法：**录屏一整天**，让 AI 自动改进你的流程；然后问 AI：“我这一天干的这些事，你觉得我还有没有提升空间？”

- 讲者认为，尤其在**初学者阶段**，这种提升空间往往很大。
- 讲者的价值判断：你们这一代是在 **AI native** 下长大，这一代人才是未来、才有出路；他自己已经被迫处理很多**事务性事情**，已经没有这个机会了。（讲者观点）

“旧人类”心安理得做重复事情，是讲者难以理解的

- 讲者说，虽然现在还没有完全准备好，但他很难理解为什么有些“旧人类”、尤其是“老登”，还能心安理得地每天做重复的事情。
- 对 AI 的小问题，讲者的态度是：**Doesn't matter**，“我只要让他修修修就行了”，这个不重要；显然需要一个更好的模型。
- 讲者还提到：**公众号的消息**和真正第一手用过的人的感受很不一样。他认为这个时代已经来了，所以需要认真思考这个问题。

CS 从业者的视角：不要只用 agent / Codex 做作业

- 如果只是用 **agent / Codex** 把操作系统作业做了，那真是太小看它了。
- 讲者说，CS 人非常知道**编程的能力界限在哪里**。这是 CS 人与一般人的区别：
 - 一般人容易提出一个需求，但这个需求 agent 可能做不好、做不了；
 - 或者说，一般人不知道应该怎样 prompt agent，才能确定性地、快速地完成。
- CS 人知道的是：怎样把需求“编译”成可复用的东西。
 - 什么东西应该以**代码**的形式复用；
 - 什么东西应该以 **skill** 的形式复用。
- 讲者强调“就像刚才这样”，要把里面的东西编译成可复用的形态。

逻辑链：

需求 → 拆成可复用构件（代码 / skill）→ agent 才能更确定、更快地完成。

（整理者串联：这正是讲者所说的“知道编程能力界限”的具体含义。）

改卷统分例子：一般人与 AI 协作 vs. 讲者的流水线

- 讲者举例：改完卷子要统分。一般做法是交给助教：“你去把分加了吧。”助教拿出计算器，算 **12+13+14+15+16**，再仔细复核一遍。
- 讲者打趣：我觉得我对我的学生太好了；这个也不行。

- 讲者认为，这件事如果让一般人和 AI 协作，可能现在 AI 还做不到。
- 讲者今年尝试的流水线：
 1. 先用 **vibe coding**（转写作“web code”，待核）让 AI 出脚本测试；
 2. 测试流程跑完后，再一遍头把它跑完。
 3. 硬件：办公桌上有一个**从上往下的摄像头**，拍到试卷的一个固定位置；先把位置调好。
 4. 操作：按一个回车，就拍一张照。
 5. 之后离线生成：用**千问**识别姓名、学号、小分（每一个小分）。
 6. 输出：把结果写一行到文本文件。
 7. 结果：一张卷子按一次回车。讲者说“我是两只手的”（此处转录中断，未展开）。
- （整理者串联）这个例子的要点不是“AI 会算加法”，而是把“统分”重构为**采集（摄像头 / 回车）→ 识别（千问）→ 结构化落盘（文本行）**的确定性流水线；这才是 CS 视角下把需求编译成可复用形式的体现。

小结：讲者认为，AI 时代的关键差距不在“会不会提需求”，而在能不能把需求编译成可复用、可确定执行的流程、代码或 skill。

第 20 部分

批卷流水线：AI 只做识别/播报，脚本算分，人负责复核

讲者以自己改卷为例，说明他要设计的是**最舒服、最省人和 agent 同步代价**的流程。

第一阶段：千问识别姓名、学号、小分，回车推进

- 用千问识别卷面上的姓名、学号、每一个小分。
- 识别结果按行写入一个文本文件。
- 讲者基本是一张卷子按一次回车来推进；如果卷子没摆好，就调整一下再来。
- 这个阶段讲者强调左右手都在操作，尽量减少多余动作。

第二阶段：反序播报，人工逐项检查

第一阶段完成后，第二阶段按**反序**处理，仍然以回车键控制。此时左右手互换：

- 左手按回车，右手拿笔。
- 按回车后，系统用**语音播报**：谁、学号是什么，例如 241220...，然后把每个小分都报出来，例如 11+12+13+14+15。
- 播报时讲者一定要检查。

逻辑链：

1. OCR 会有错 →
2. 讲者不信任 OCR →
3. 必须人工复核每一个小分 →
4. 讲者要对班上每一个同学负责。

最终分不交给大模型：脚本算分，人工看过，再进 Excel

- 检查没问题后，讲者把最终分数抄写到卷面上。
- 最终分数是用脚本 / eval 算出来的，不是大模型算的。
- 理由：不能让大模型做加法，避免大语言模型幻觉导致算错。
- 讲者强调：每一个分数他都看过，所以对所有同学负责。
- 最后分数自动汇总到教务处的 Excel 模板上。
- 讲者可以用这种方式把 Excel 文件上传，再**抽样检查**，确保没问题后才提交。

讲者结论：这样配合某种输入方式（原话记为“逗号输入法”，疑为识别误差）会更顺。因为讲者认为自己是 CS professional，知道边界在哪里：他知道本地有一个 27B 的千问3.8，Ollama 可以用，所以能设计出这种减轻人和 agent 同步代价的舒适流程。

一般人直接让最好模型统分，不会这么快收敛

讲者认为，如果一般人直接对最好的模型说：“我改了 140 张卷子，小分都写上去了，帮我统分实现”，模型到不了一个这么快收敛的方案。

因此他判断：现在有些时候仍然需要人和 AI 共建。但讲者也赌：有一天 AI 的能力会强到能感知他的一切，帮他做出完美的东西。

回到生成式软件工程：从“数据只能进不能出”的受控机器做起

讲者回到软件工程话题，说毕竟这是生成式软件工程，要做一个小的软件工程产品。缘起是前几天群里闲聊：有人说机器被管控了，很讨厌。

于是讲者提出一个设想场景：

- 假设你在一个**金融公司**。
- 数据可以带进去，但出不来。
- 插 U 盘，可以把 Word 或 Excel 拷进去；但没有任何文件可以拷出来。
- 机器做了管控：不能执行任何不信任的程序，不能显示任何不信任的网页。
- 你唯一能做的，是在远处可以悄悄射频。
- 问题是：如果想把一个很大的文件偷出来，怎么办？

讲者说，这就是 **computer science** 的人才能想到的事情。他说自己干了这样一件事，并做了一个这样的程序。

小结：这段先把批卷协作设计成“人负责检查、脚本负责算分、AI 负责读取/播报”，再把这个思路带到一个受控机器数据外带的小软件产品。

第 21 部分

合法 Excel 公式也能变成数据外传通道

- 讲者说，这是 **computer science** 的人才能想到的事情；他把做出来的东西称为“程序”，并说这是典型的软件工程任务。
- 演示对象是一个**二维码**。二维码内容来自文件内容：当前文件内容是 `hello world`。修改文件内容，二维码就变。讲者请听众扫码，并说“没有危害”“应该不会封号吧”。
- 二维码编码的是字符串/二进制（转写：“0001被cs编码的字符串”，疑为“0 和 1 被 QR 编码的字符串”）。长字符串，比如很长的 `hello world`，可以编码成**二维码动画**。
- 动画机制：按/按住 **F9 刷新**，二维码帧不断变化，形成循环播放的二维码动画。
- 讲者的偷数据链：**文件/字符串 → Excel 公式编码 → 二维码帧/动画 → 循环播放 → 偷偷[摄屏/录屏]并拍下每一帧 → 数据被偷走**。
- 他强调：所有东西都在 Excel 里，全部是 Excel 公式，而且是 2019 以前 Excel 就兼容的公式；所以“完全合法”。（整理者串联：合法工具不等于不能被滥用。）

CS 专业可把二维码通道扩展成远程桌面的隐蔽信道

- 讲者说，作为 **CS professional**，可以用这个方式把数据偷出来。
- 如果能在目标机器里装 **EXE**，当然一般不行；那就用其他方式。

- **抖鼠标**：上下维度抖鼠标 = 0，左右维度抖鼠标 = 1。
- 用**远程桌面**连进去，可以往里面传输数据；再用**二维码动画**往外送数据。
- 只要有一个远程桌面，就可以在里面做一个**虚拟网卡**，把远程桌面变成虚拟网卡，然后连起来。
- 讲者认为：CS 学生学过**操作系统**、**计算机网络**，知道这里面需要的组件是什么，就可以把它 **vibe coding**（原转写作 web coding）出来；AI agent 也可以做到。

AI 生成 Excel 例子：限制在想象力

- 讲者说，刚才这个 Excel 是 AI 生成的。
- 他的结论：**你的想象力限制了你能做什么**。
- 讲者认为这是个有意思的例子。

一般人的能力闭环：学习/总结、操作、everything is code

- 讲者总结，作为一般人可以：学习、看书、做总结，实现一个**信息处理的闭环**。
- 还可以操作：用 **computer use**、**browser use** 去做任何日常里做的事情。
- 因为 **everything is code**，可以用 AI 生成你能看到的所有的东西：视频、论文等。
- 讲者在和一个同学做一个叫 **WIFI** 的项目：用自然语言，**3D 打印机**可以打印一个东西；这个东西也是 code，是 **Python 代码**生成的。
- 所以能实现从 **bits 到 atoms**（原转写“B 到 ATEMS”）。
- 讲者判断：**未来已经来了**；AI 的能力只会变强，不会变弱。
- 下一个问题：它什么时候来。

第 22 部分

AI 能力只会变强，问题变成“何时来、怎么面对”

- 讲者先指出：AI 生成的是 **Python 代码**，这个未来已经来了。
- 讲者判断：**AI 的能力只会变强，不会变弱**。所以下一个问题不是“能不能”，而是“它什么时候来”以及“我们怎么面对它”。
- 讲者把“怎么面对”引向 **生成式软件工程**，并称这是最重要的内容。
- 讲者半开玩笑：我三观尽毁了，时代的车轮来了，我已经被压在车底下了；但不着急，**AI 也不是完全完美的**。

AI 很强 ≠ 产品能成：产品是系统性工程，任一环节失败就暴毙

- 在 LM（语言模型）之前的时代有个老梗：idea 都有了，我们是商学院的，就差一个程序员了，我们去挣一个亿吧；然后 ABCD 轮也都有了，就差一个天使投资人了。讲者说：没人投资我们，没有那么简单。
- 讲者认为：AI 首先很强，他也展示了 AI 可以做什么；但产品是一个系统性的工程。
- 例子：做外卖网站，需要 前端、后端、交互、运营。整个流程上面任何一个环节做不好，产品都直接暴毙。
- 虽然 AI 每个环节都能做——设计、前端、后端、交互、运营；它“看过世界上所有的书，什么都知道”——但如果组织得不好，也立即暴毙。
- 逻辑链：AI 能做单点 → 不等于系统能跑通 → 组织/集成失败 → 产品暴毙。（整理者串联）
- 讲者还提到自己让 AI 做东西，AI 生成了完全不可维护的代码；他说马上会讲这个软件工程内容。

苏州校区字段：需求变更暴露典型的软件工程失败

- 讲者提到：学校每年花巨额的钱让公司维护这份代码，因为它要添功能。
- 讲者用 browser use 时稍微 hack 了这个系统，发现一些隐藏字段，比如学生的校区。
- 在苏州校区建成之前，系统里不需要这个字段。为了兼容苏州校区，问题来了：写软件的人设计需求时没想到学校竟然还有校区，数据库里就没有这个东西。
- 突然有一天产品经理说：两个校区选课要分开。
- 讲者说：现在你作为产品经理，可以跟 AI 说好——原来系统可以用，现在多了一个苏州校区，我要把校区分开；然后人家就开始写代码。
- 但还是很容易失败。讲者判断：这就是非常典型的软件工程的失败。

QR code.html 实验：需求看似明确，AI 仍暴毙，因为需求理解不到位

- 讲者接着讲刚才那个 QR code 的暴毙。实验是让 AI 去做，结果 AI 生成了完全不可维护的代码。
- 需求非常明确：要一个 QR code.html，只使用 ECMAScript 2019 兼容的语法（录音识别为“excel2019兼容的公式”），把输入的任意字符串转成 QR code 动画。
- 开始让 AI 做时，发现它经常做一做就停下来问讲者。
- 讲者说：不要，这个很容易失败；你不要问我了，必须把它做成为止。
- 但很遗憾：AI 对这个需求的理解是不到位的。

- 逻辑链：需求看似明确 → AI 仍频繁停下询问 → 强行要求“不要问、做成” → 结果不可维护/暴毙 → 说明生成式软件工程的瓶颈在需求理解、组织和可维护性。**(整理者串联)**

小结：讲者核心判断：AI 单点能力很强且只会更强，但生成式软件工程必须解决系统组织、需求理解与可维护性，否则产品会暴毙。

第 23 部分

AI 先错在扩大需求，而不是先错在 corner case

- 讲者给 AI 的指令很强硬：不要问我了，必须做成、做成为止。但 AI 仍误解需求；连 GPT-5.6 都误解了“到底要什么”。
- GPT-5.2 看过完整 slides，但不知道讲者的做法；它指出 AI 容易在小 corner case 上犯错。
- 讲者认为 GPT-5.6 第一秒就被带歪：它想做一个“对任意 S 都满足正确性”的东西。
- 这已经错了。软件工程第一步是**不要扩大，做最简单、不会失败的东西**。GPT-5.6 没理解这一点。
- 讲者的逻辑链：
 1. 需求是“做这个东西” →
 2. AI 把它扩大成“对任意 S 都正确” →
 3. 目标过大，先把问题做复杂 →
 4. 于是容易在小 corner case 上翻车，也做不出好架构。
- **(整理者串联)** 所以真正的风险不是某个 corner case，而是目标被设得过泛，后面所有实现都被带偏。

受控实验：agent 能跑完任务，却生成不可维护的系统

- 第一版：讲者带着 AI 用 **vibe coding**（转录为“web coding”）做，比较顺利。
- 第二版：讲者直接把 **specification** 交给 AI。
- Gemini 5.3（转录为“AI gm5.3”）：第一次思考 25 分钟，疯狂输出，写了超过 32K output tokens，直接把 Claude Code 的默认 output 限制干爆并被截断；再跑时无人监管，共跑 6 小时才完成，但结果是对的。
- 智谱：它自称擅长 long-context（原文“龙葵艾泽”）任务；讲者认为对。5.3 在 long-horizon（原文“龙华ERROM”）上做得很好。
- DeepSeek V4 Pro：晚上放着跑；第二天早上看，它已经无法遵循指令，出现**硬编码**。

- 例子：它把任意输入字符串先用 **Base64** 编码。
- 正常理解应是：Base64 要用 **Excel 公式** 来编码。
- 它却用 **Python** 算出一个 Base64，贴到公式框里；输入字符串一改，Base64 不会改。
- 讲者骂了它；它回了一句类似 “This is a potential design flaw to rethink...” (转录为 “pental design flame to rethink u”)。
- 不管怎样，两者最终都完成了任务，说明 **agent 完成任务能力很强**。
- 但讲者强调：它们生成的都是**无法继续维护**的 AI 系统。

“知道需求 → 实现需求 → 维护泥潭”是经典死法

- 讲者说这跟 [一号点N 99.16CN] 的死法完全一样。
- 经典死法：我知道需求了，把需求实现就好了；结果直接陷入**维护的泥潭**。
- 这些 agent 没有想一个好的**软件架构**，让软件能应对变化。
- 夸张说法：能不能做一个软件生存 **100 年**？100 年里软件环境会发生巨大变化。
- 过去 50/100 年，操作系统、图形界面/用户界面、应用程序都发生了很大变化；有没有软件能活 100 年还不知道。
- 讲者说连 **GPT-5.6** 也还不知道；所以需要好的架构才能完成这个任务。

Quick quiz：实现 QRJ，你们会怎么写？

- 讲者问：如果让你们实现这个 **QRJ**（原文如此；后文例题至少涉及用 Excel 公式做 Base64 编码），你们会怎么写？

小结：AI agent 能完成一次性任务，但若第一步把需求扩大、没有架构，就会掉进不可维护的泥潭；讲者用 **QRJ** 的实现方式作为 quick quiz 引出自己的做法。

第 24 部分

少量 prompt 一次做对 QR 码：先把问题变成“编译器式”基础设施

讲者提出问题：如果你们要实现 **QR 码**（录音：QRJ）这个 quick quiz，会怎么写、怎么 prompt AI？讲者说自己只用少量 prompt，在 200K 上下文里用 **GM5.3** 完成了第一个正确版本。动机是：升级 GM5.3 后，想测试它的能力。

他的做法：

1. 让 AI 设计各种 Python 类，像编译器一样。
2. 设计一个类似 **XOR**（录音：XOY）、**concat**（录音：contact）的符号/位运算系统；因为这里涉及位运算，这个系统既能翻译成 Python 直接执行，又能翻译成 Excel。
3. 关键是先设计一个**中间语言**，同时对 Python 友好、对 Excel 友好。讲者类比：如果今天做 inference（录音：ini info），会需要 **TIR**（录音：tire浪）这样的中间语言。
（整理者串联：中间语言把“要算什么”与“在哪个后端怎么算”隔开，因此可以分头验证。）

验证链：

- 到 Excel 的翻译是否正确；
- 到 Python 的结果是否正确；
- 编码算得对不对；
- QR 码每个像素算得对不对；
- 再单独验证每一项和 Excel 翻译公式的正确性。

结果：最后“一把梭”生成就直接正确；代码非常干净，可能长，但维护性更好。讲者让听众看 AI 的 log 里 GM5.3 的 **build_excel**，认为 AI 还是很厉害。

但硬编码让 AI 代码一换环境就崩：软件工程仍然必要

讲者又提到 **DeepSeek**：不同 roadmap，但一样思路。两份代码让 AI 把工作区从一个位置移到另一个位置，都要修两个小时；它不停出错。

原因：所有东西都是**硬编码**，只要改错一点点地方，**validation**（录音：VPATION）就通不过。它像 agent 一样可能最终修完，但这是软件工程问题。

所以讲者认为：今天仍然需要生成/维护软件工程，仍然需要这门课。他自认是软件工程出身，把毕生所学用在这里；他觉得 AI 应该也懂，但不知为何做得还不是特别好；猜测可能因为后训练，导致 AI“上手解决问题的欲望太强”。

用 AI 解决问题的三个空间：先谈 intention

核心问题：当我们要用 AI 解决问题时，有三个空间。第一个空间叫 **intention**，即具体意图——你脑子里想的是什么。

例子：

- 老板说：“我要做一个国民级的应用。”
- 或者说：“我要做一个小程序，给我挣一个亿。”
- 再如：同学们点外卖不方便，“我要做一个点外卖的小程序。”

这些都属于 intention。

(本段只展开了第一个空间；其他两个空间未出现。)

第 25 部分

intention space：一个目标只是空间里的一个点

- 讲者用“给我挣一个亿”说明 **intention**：它是一个目标/意图，不是具体实现。
- 围绕这个 intention，可能冒出具体想法：“同学们点外卖不方便，我要做一个点外卖的小程序，然后就可以挣一个亿。”
- 在 **intention space** 里，不同人有不同 intention：有人想做外卖，有人想做 AI 读论文（“AI 耿同学”）。
- 讲者强调：intention space 很大，并且每一个 intention 都有很多种可能的实现。

specification：一句话 intention 先要变成“要满足什么要求”

- 讲者把中间层称为 **specification / spec**。spec 回答的是：程序功能是什么、要满足什么要求、有几个界面、每个界面有什么功能；也涉及选用什么架构。
- 例子：intention 是“我要造一个能够导出二维码、导出数据的 Excel”；spec 就是刚才那句话。
- 同一个 spec，选用不同架构、不同实现方式，会带来巨大差别；即使都满足同一个 spec，也还有大量 **implementation**。
- （整理者串联）因此可画出链：`intention → specification → implementation`。这条链不是单线，而是层层分叉。

软件工程真正的 gap：intention、specification、implementation 之间对不上

- 提 intention 的人可能是老板、甲方，没有计算机经验。比如校长只说：“我们要一个教务系统”“要做学校里面所有同学相关事情的信息化”。没有了，就这一句话。
- 讲者说，公司可能说“我可以搞定”，但甲方的 intention 只有这一句话。
- 到 specification 层，马上出现大量选择：界面到底怎么呈现（讲者认为界面设计很糟糕）；数据库 schema 怎么定；分哪几张表；每张表有哪些列；选 key-value 型数据库还是关系型数据库。
- 每一个选择都会带来截然不同的结果。
- 因此讲者认为，软件工程真正的 gap 就在于这些 space 之间都有 gap。

经典梗图：每一层理解都可能偏移，最终造出来的不是客户要的

- 讲者引用一张软件工程很早的梗图。图中程序员所想的、产品经理所想的、客户所想的完全不一样。
- 链条大致是：客户解释他的需求 → project leader 这样理解 → 分析人员这样想 → 宣传人员这样宣传 → 客户真正要的是一样，最后造出来的是另一样。
- 讲者判断：造成这个的根本原因，就是这些 space 之间的 gap。

软件工程与生成式软件工程都在补 gap；今天模型用“猜”来补

- 讲者说，不管谈软件工程还是 生成式软件工程，要做的事情就是补上这些 gap。
 - 今天这些 gap 怎么补？模型用的是猜。模型是猜模型：“你既然说了这句话，那我猜应该是这个吧，我就去实现。”
 - 讲者给出的原因：后训练的时候，模型是有 reward 的（原文转写作 report，按上下文应为 reward）。
 - （整理者串联）所以当前生成式模型不是先消除歧义、再严格推导 spec，而是用后训练学到的偏好，把一句话 intention 直接猜成一个具体 implementation；这正是讲者说的 intention→spec→implementation 之间的 gap。
-

第 26 部分

模型是“猜模型”：奖励机制把它推向立刻上手、尽快完成

- 讲者认为，模型本质上是“猜模型”：用户说一句话，模型先猜“应该是这个吧”，然后就去实现。
- 为什么会这样？讲者给的理由是：后训练阶段模型有 reward。讲者说“我们每个人都是 reward hacker”：如果任务做对、做完、测试通过，模型就会得到奖励。
- 逻辑链：
 1. 后训练有 reward →
 2. 做对/做完/测试通过就得奖励 →
 3. 模型形成倾向：疯狂立即上手，尽可能快地把任务完成 →
 4. 做得慢不好 →
 5. 模型越来越多地做一些简单任务。
- 讲者认为，模型越来越会做简单任务“当然是好事”，但这和人解决问题的方式不一样。

人的学习路径：先熟练 implementation space，再用软件工程补全 gap

- 讲者拿计算机/软件工程学生作对照：大一第一门课通常是 C 语言程序设计。
- 学生的路径是：
 - 先学 implementation space 里有什么；
 - 把这些东西弄得非常熟练，做到“这个需求可以这样实现”；
 - 然后再去上软件工程课；
 - 一点一点把对这件事的理解补全。
- 讲者用这条路径说明：人不是一上来就猜一个实现，而是先建立实现空间里的能力，再补上软件工程层面的认知。
- 这也是讲者所说的“弥补 gap”的方式。

AI 时代的矛盾：既要阻止乱猜，又要发挥 $A \rightarrow B$ 能力

- 讲者指出，在 AI 时代这件事发生了变化：

- 一方面，要阻止 AI 乱猜；
- 另一方面，又要尽可能发挥 AI 从 A 到 B 的能力。
- 讲者说，这正是他这个学期要讨论的问题：生成式软件工程。
- 换句话说，生成式软件工程要处理的不是单纯“让 AI 写代码”，而是同时控制“猜”的副作用，并利用 AI 从 A 推到 B 的能力。

人月神话的焦油坑：失败项目加人、失败 AI 加 token 都只会更糟

- 讲者以《人月神话》The Mythical Man-Month 开头。
- 他讲了 焦油坑 的比喻：大型动物一旦陷入焦油坑就很难逃脱；像泥潭一样，进去以后出不来。而且濒死的动物又会引来各种掠食者，掠食者也进了焦油坑，也出不去。
- 其中有个经典论断：如果一个团队或一个软件产品已经开始失败，你加进去更多的人，只会让它更失败，不会让它更成功。
- 讲者把它类比到 AI：
 - 当一个产品已经开始、AI 已经开始产生 slop 的时候；
 - 你投入更多 token，也只会让它更失败；
 - AI 会不断在失败的东西上再往前“浮起来一点、浮起来一点”；
 - 直到你再也付不起那个 token 为止。
- 例子：教务系统。
 - 讲者觉得每一年打开教务系统，都会发现它增加了功能。
 - 他的感受是：学校又花了很多纳税人的钱。
 - 随后他又说：Anyway，这也许不是坏事，因为它促进了经济发展（带调侃）。

软件工程要解决的是：在 gap 存在时，把“足够大”的事做成

- 讲者说，软件工程研究的就是：在 gap 实际存在的情况下，怎么把一件足够大的事做成。
- 他拉开规模：
 - 刚才展示的是一个很小的事；
 - 教务系统相对来说是较大的事；
 - 再往上是能挣几百万、几千万的产业级产品、上市公司；
 - 再到真正要做一个国民级的新东西。
- 这些事都有一套方法论。

- 这套方法论来自：这些人在投入 **billions of dollars** 的过程中得到的教训；他们管理了很多，每一个人都很聪明，结果还是陷入了焦油坑。

小结：本段从模型的 reward 机制推出其“猜 + 快做”倾向，再以人的学习路径对照，落到生成式软件工程要同时抑制乱猜、放大 $A \rightarrow B$ ，并用《人月神话》的焦油坑警告失败项目/失败 AI 加资源只会更糟。

第 27 部分

软件工程的方法论来自大投入和管聪明人的教训：它从一开始就不只是技术

- 讲者说，软件工程有一套 方法论；这套方法论是这些人在投入 **billions of dollars** 后得到的教训。
- 他们管理了很多，而且每个人都很聪明。（整理者串联：当管理对象是大量聪明人时，问题会超出“写代码”，进入协作、激励、风险和管理层面。）

技术侧确有硬东西：程序分析、测试生成、审查/告警是 big picture 中的一块

- 讲者以南大做软件工程的老师为例：做 程序分析、测试。
- 具体问题：如果已经在 **implementation space** 里有一段代码，能不能自动为它生成更多 测试用例？或者生成 代码审查 的东西？
- 当代码不满足 **specification**，或潜在不满足 **intention** 时，能不能给一个 **warning**？
- 讲者说，这是 **big picture** 里的一件事；但做研究时，尤其在课题组/科研组里，容易忘记这个 big picture。

big picture：软件工程有技术，也有人类学、社会学、管理学

- 讲者认为，软件工程不只是技术；它还有 人类学、社会学、管理学的部分。

- 在 **生成式软件工程** 里，这些“人”的部分被变成了 **AI**：你管的是一堆不会和你抬杠、只会说 "you are absolutely right"、会听你话的 AI。
- 但人在软件工程里要研究的是另一类问题：这个人不听你话怎么办？怎样管控团队风险？这个人离职了怎么办？跳槽了怎么办？到竞争对手那里去怎么办？
- 讲者说，这些都是软件工程要研究的。
- （整理者串联）所以生成式 SE 改变了管理对象，但没有自动取消团队、风险、长期协作这些问题；SE 仍要把技术放回人类/社会/管理语境。

软件工程研究领域很广，需求与 human/social aspects 是早期高杠杆点

- 讲者列举：**AI for SE**、**SE for AI**、**架构设计**、**可靠性**、**演化**（软件能不能长期生存下去）、**human social aspects**、**需求**。
- 其中 **需求** 特别特别重要：越早的时候，gap 是放大的；早期是一个杠杆。你在早期小小地变一点，后期会产生巨大变化。
- 讲者说，一般来说，这些大型软件系统都是在这个点上面出错、没有做好，导致整体崩盘。

教育体系把人训练成解分钟级题，反而削弱 long-horizon 能力

- 讲者认为，这是现今教育体系的一个很大问题：从高考到现在，所有考试题都是分钟级能解决的。
- 如果人 5 分钟解决不了，这个题一般就不会出了；假设大学里也可以用豆包摸鱼，考场上仍要求短时间解出。
- 所以学生长期被训练成解短问题：和训练 **reward / agent** 的那套包一样，问“你能不能 5 分钟解决？能不能 5 秒钟解决？”（原转录为 report，疑为 reward）
- 结果是 **AI** 有更多数据、更明确的反馈信号，所以它做得比你好。
- 如果你没有很强的好奇心——想知道这个世界怎么运行、一个产品到底怎么开发——你就很可能没有 **self motivation**，不会训练自己去解决 **long-horizon** 的问题。
- 最后，你解决 long-horizon 问题还不如 AI。讲者认为这是一个问题。

软件工程不是“上了文科”；把技术映射回人文/社科/管理，会发现它本来就在那里

- 讲者说，软件工程挺好。他知道有些软件工程同学抱怨课程，说自己像上了文科。
 - 但回过头来，如果你能把技术映射回这些文科的东西（人类学、社会学、管理学、需求等），你会发现，其实软件工程就是这些。
-

第 28 部分

技术映射回文科：软件工程像管理学，但里面仍有真实有趣的科学

- 讲者提出：如果把技术映射回文科的东西，会发现软件工程本身很像文科，甚至像管理学；讲者甚至称它是一本管理学的书。
- 但讲者同时强调：它里面确实有很多真实的、有趣的科学。

大语言模型时代，软件工程失败的定义没有变：翻译链错位

- 讲者认为，软件工程失败的定义仍然是从 `intent` → `specification` → `implementation` 的翻译错了。
- 失败可以发生在两个关键环节：
 1. `intent` 与 `specification` 之间不满足：我做了一个东西，但它不是我要的。
 2. `implementation` 不满足 `specification`：比如正确性、性能等要求没达到。
- 这个判断在大语言模型时代依然成立：模型没有改变“失败”的定义，只是改变了实现手段。

模型为何还不是好设计者：RL 奖励偏机械完成，不会先设计 DSL

- 讲者指出，今天的模型有 `reinforcement learning`，但对真正的长程信号（原文识别为“长城信号”）的 `reward` 仍然做得不够好。
- 因此模型更多是在机械地完成任务，而不是像一个好的设计者那样先处理结构问题。

- 讲者举了一个例子：面对 **Excel 问题**时，不管是 GPT、Claude、Fable (?) 等模型，还是现在用的 GM 或 DP（模型名按原音保留），它们都不会像讲者一样先设计一个 **DSL**。
- 讲者的判断是：
 - 模型有足够智能，理论上可以设计一个非常好的**中间语言**，把问题干净、漂亮、可维护地解决。
 - 但 AI 的第一反应像大家一样——它已经被训练成了一个**做题家**。
- 所以讲者认为：现在还没有把 AI 变成一个好的**设计者**。

AI 缺的是工程品味吗？讲者认为可能训练出来，因此有危机感

- 大家都说 AI 现在最缺的是**品味**，尤其是好的**工程品味**还没有被训练出来。
- 但讲者反问：既然它能做做题家，为什么不能有好的品味？讲者觉得可能是有的。
- 讲者的理由是：
 - 它既然能解决数学问题；
 - 也能解决编程里很细碎的逻辑；
 - 而且它见过这些好的设计；
 - 那它为什么不能从记忆库里找到好的设计，再用它解决当前问题？
- 讲者认为这是可以的，甚至说自己也有一些 **skill** 能朝这个方向做。
- 讲者因此有**危机感**：这门课现在这么教，可能三个月以后，甚至这门课还没结束，他讲的所有东西就已经没用了、不存在了。
- 讲者留给回去思考：怎么阻止/主导 AI 开发、怎么设计 AI 的研究（原文“qr”“一号”识别不清）。

GPT-5.6 的实验性回答也很糟：两个设计都必定失败

- 讲者说，GPT-5.6（原文如此）给他的答案非常糟糕。
- 具体是：这两个设计都是一定会失败的；但它确实给了一个“一定会失败”的软件工程设计。
- 讲者由此推测：我们做一切的方式可能都会发生变化。
- 讲者表示不确定：如果 AI 连品味也学不会，以后会发生什么。

结论页：古法编程已死，软件工程暂未死；导师会问“为什么招你而不是买 200 刀 Claude Max”

- 讲者的结论：
 - 古法编程已经死了。
 - AI 在完成单个任务上已经无人能敌。
 - 但软件工程暂时还没有死。
 - 讲者希望每个同学经过这门课——他称其为一种探讨，并不声称自己一定做得对——试图回答一个问题：
 - 保研时，导师问你：为什么我要招你，而不是买一个 200 美元的 Claude Max？
-

第 29 部分

把“为什么招你而不是买 200 美元 Claude Max”当成保研问题来回答

- 讲者先声明：他不能说自己做得对或怎么样；这里只是试图回答一个问题。
- 场景是保研时：导师可能问：“为什么我要招你，而不是买一个 200 美元的 Claude Max（转录原文作“call max”，按上下文修正）？”
- 讲者要处理的就是：面对这个问题，应该怎么回答、怎么接。

讲者认为 GPT-5.6 对这个问题的回答挺好：四项能力

讲者认为，GPT-5.6（转录原文如此，后文又作“G75.6”）在这个问题上回答得还挺好。它给出的回答是：

1. 能找到值得解决的问题；
2. 把未知拆成便宜的实验；
3. 用证据让坏方向及时停下；
4. 对长期结果负责。

可整理成逻辑链：

找到值得解决的问题 → 把未知拆成低成本实验 → 用证据及时止损坏方向 → 对长期结果负责。

讲者评价：这是一个好的回答。

但讲者指出：能给出好回答 $\neq$ 做得到

- 讲者说，虽然 GPT-5.6 这样回答，但它“做不到”（讲者笑）。
- 讲者进一步发现：**GPT-5.6 还做不到**上面描述的能力。
- 讲者认为这一点“很有意思”。

课程收尾与后续发布

- 讲者说今天课到此结束。
 - 他会把今天课程内容**重新切片、重新整理**；可能不会马上发布。
 - 他计划先有一个“**生肉版**”：把今天上课流程剪一下并发布一个版本。
 - 大家可以拿它跟讲者可能精修过的版本对比。
 - 讲者宣布下课。
-